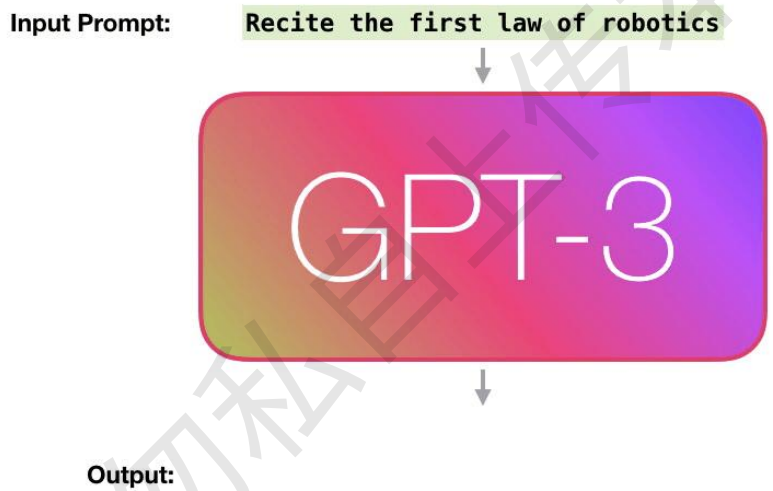
生成式计算与大模型



请勿私自上传本课件

计算机与人工智能学院
马冲

生成式计算与大模型

1.1 ChatGPT与大模型简介

1.2 Transformer与大模型计算原理

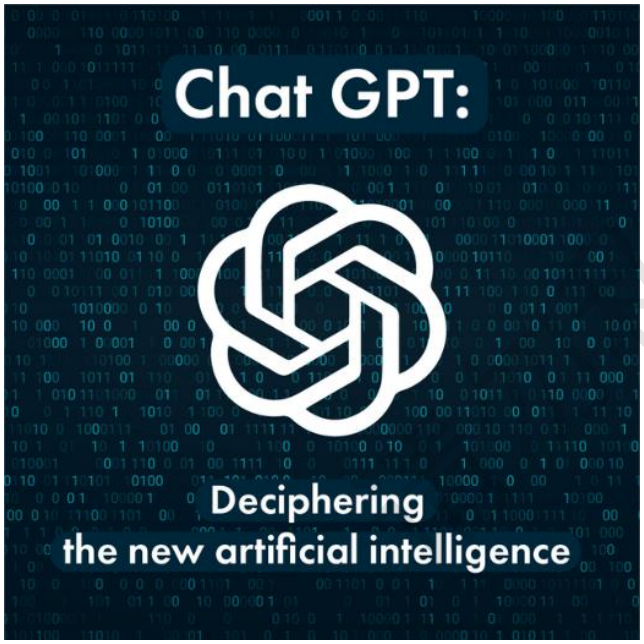
1.3 Langchain与大模型开发框架*（自选学习）

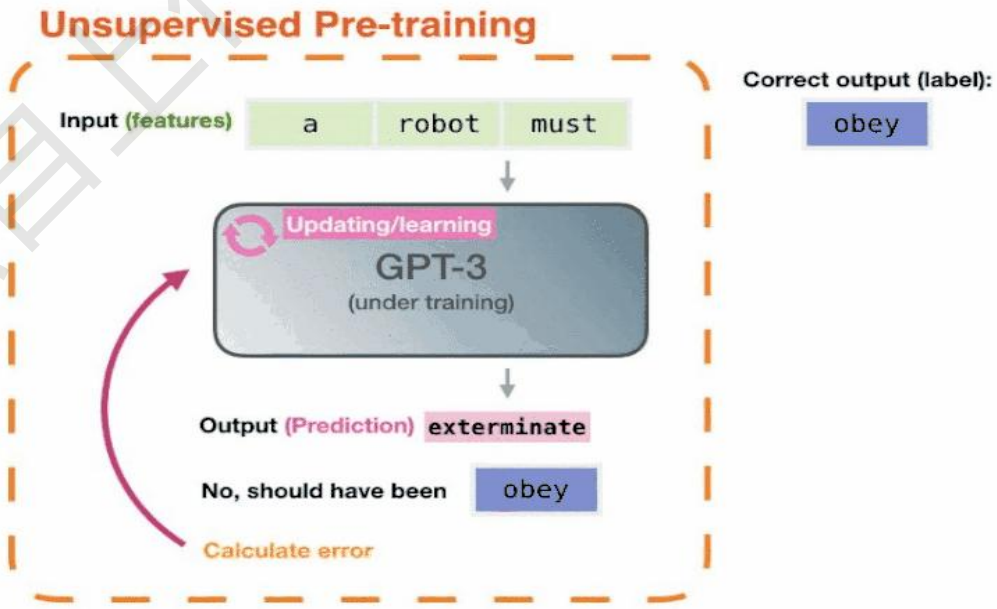
请勿私自上传本课件

1.1 ChatGPT与大模型简介

ChatGPT引领大模型AI热潮

ChatGPT 是由 OpenAI 在2022 年 11 月推出的全新对话式通用人工智能系统，它能够通过学习和理解人类的语言来进行对话，还能根据上下文语境进行互动、聊天交流，甚至能完成撰写邮件、视频脚本、文案、翻译、代码等。





1.1 ChatGPT与大模型简介

ChatGPT引领大模型AI热潮

2018.6-2020.5

2018年6月，OpenAI 提出了第一代 GPT（Generative Pretrained Trans_x0002_former）模型，将**自然语言处理带入 “预训练” 时代**。在后续两年里又陆续推出了GPT-2和GPT-3,模型的参数量也越来越大，**GPT-3参数量达到了1750亿**。

2022.3

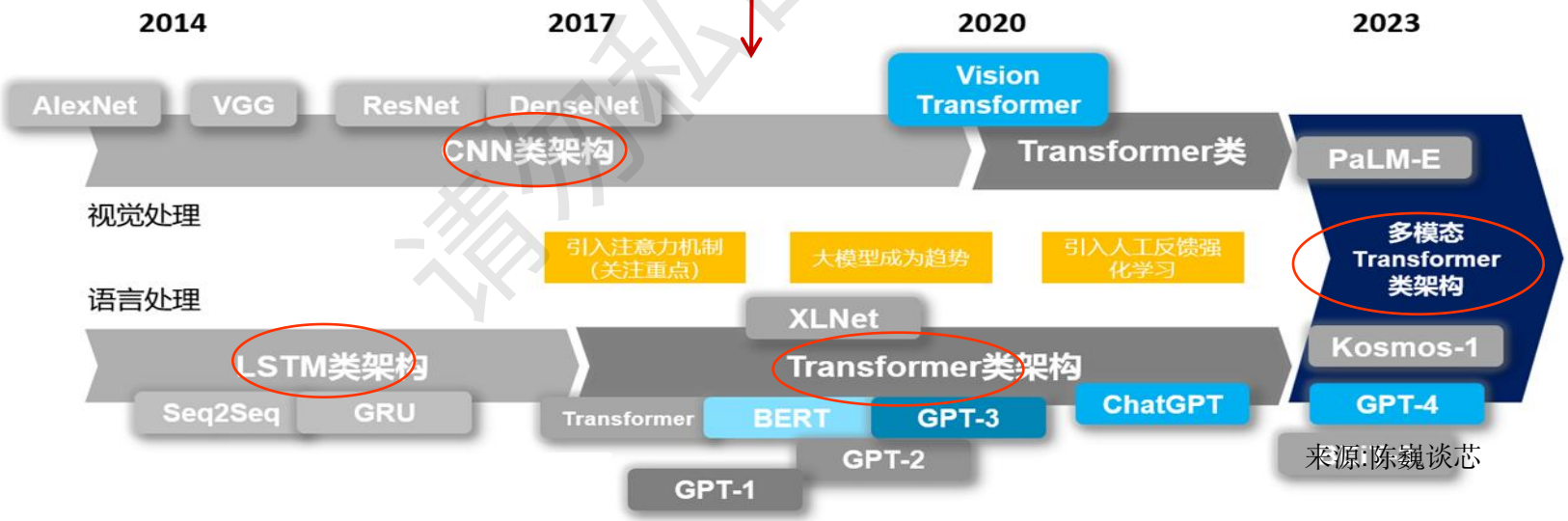
推出了GPT-3.5。在**训练过程中引入了人工反馈的强化学习（RLHF)来训练模型**，模型训练的效率和效果都得到了大大的提升。

2022.11

在GPT-3.5的基础上优化了对话，**推出了人工智能对话聊天机器人ChatGPT**。

2023.2

微软官方公告表示，旗下所有产品将全线整合ChatGPT，并推出由ChatGPT支持的最新版本的人工智能搜索引擎Bing和Edge浏览器。



来源:陈巍谈芯

请勿私自上传本课件

1.1 ChatGPT与大模型简介

ChatGPT功能特点

对话功能

ChatGPT可以进行智能对话，与用户进行自然语言交互，回答用户的问题、提供建议和解决问题。

文本分类功能

ChatGPT可以对文本进行分类，将文本归为不同的类别，如情感分类、主题分类等。

文本摘要功能

ChatGPT可以对一篇文本进行自动摘要，提取文本中的重要信息，生成简短的摘要。

语言生成功能

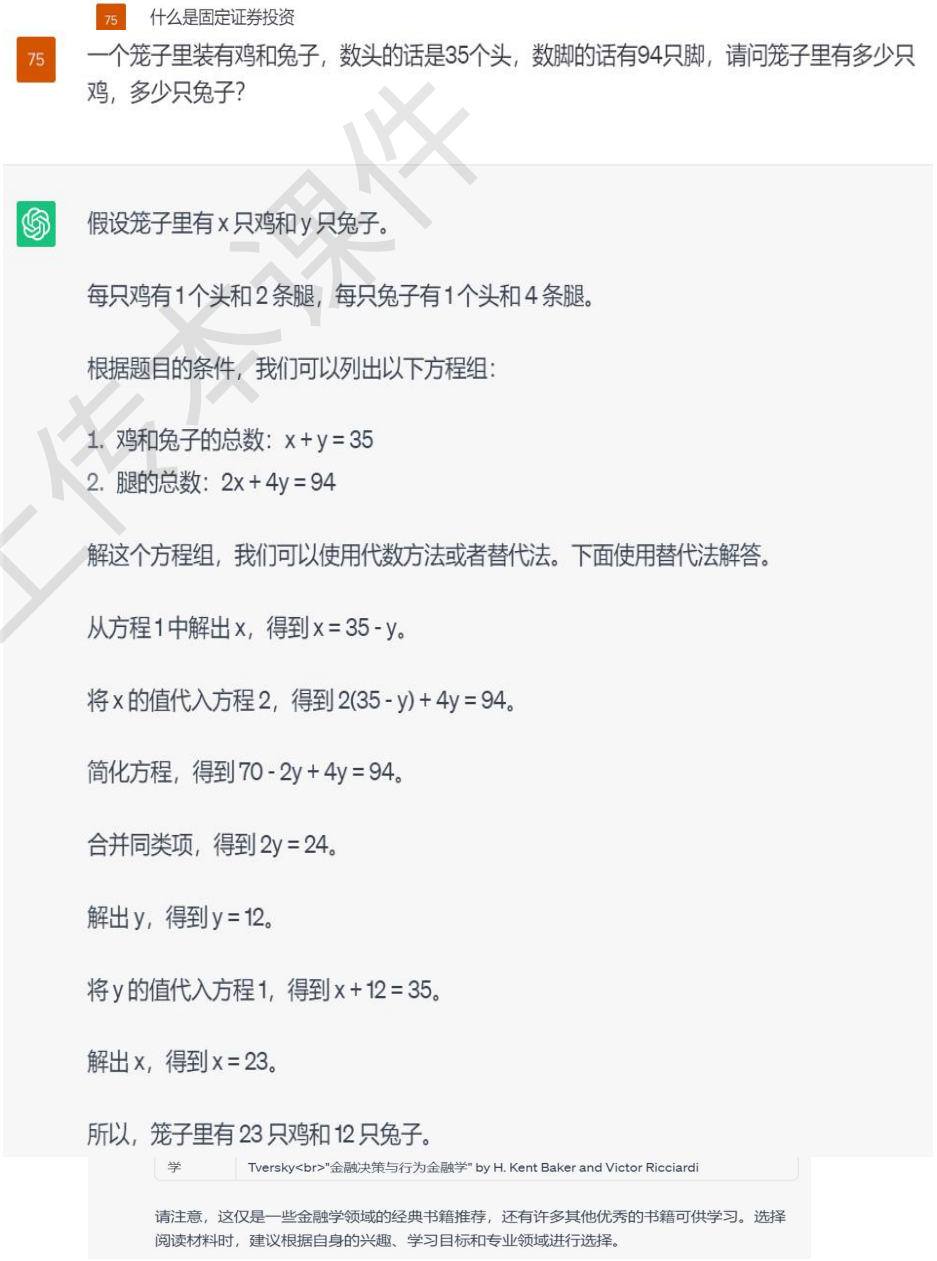
ChatGPT可以生成文本，包括自然语言描述、文章、摘要、答案等等。

机器翻译功能

ChatGPT可以进行机器翻译，将一种语言翻译成另一种语言。

其它自然语言处理功能

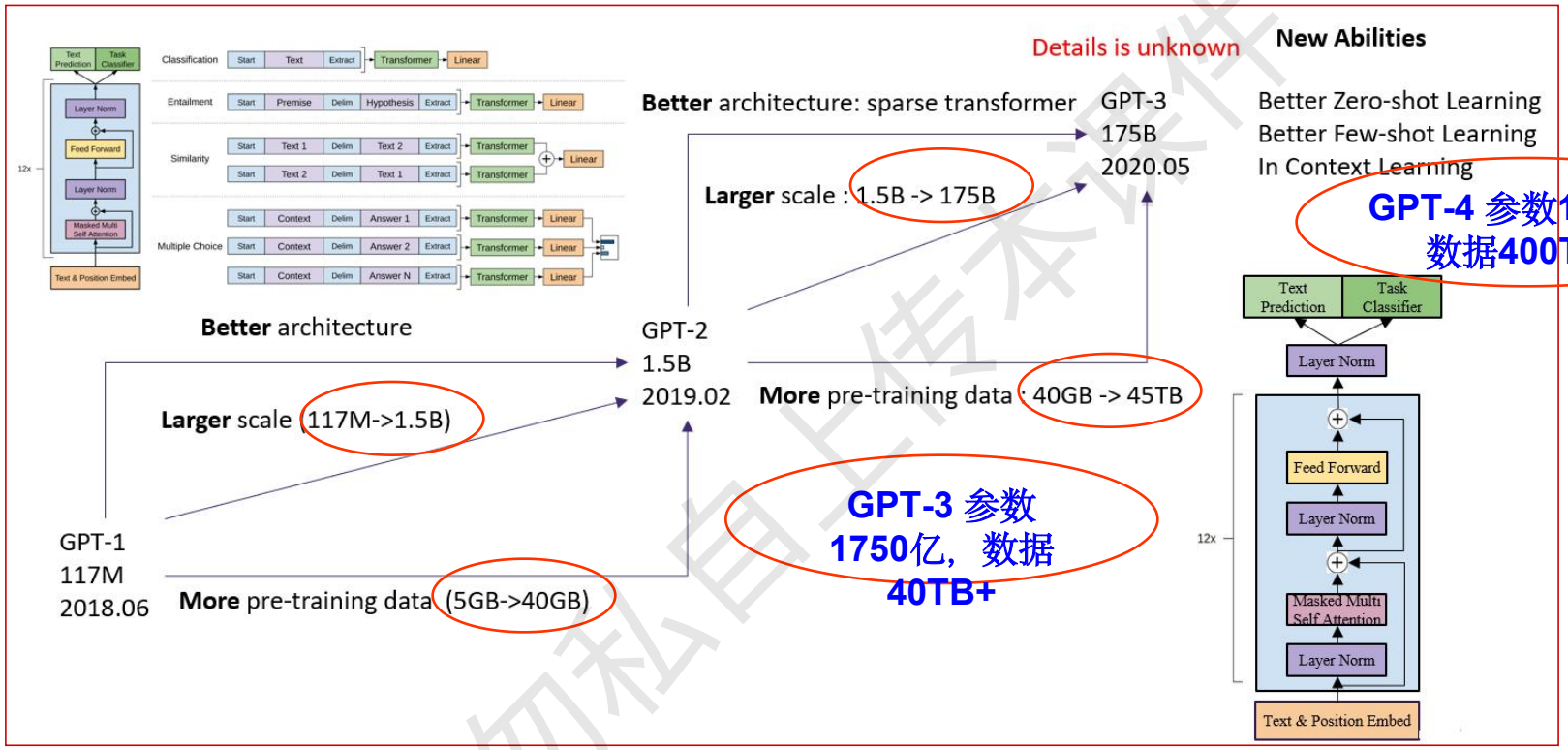
ChatGPT可以进行自然语言处理，包括分词、命名实体识别、情感分析、语言模型训练等。

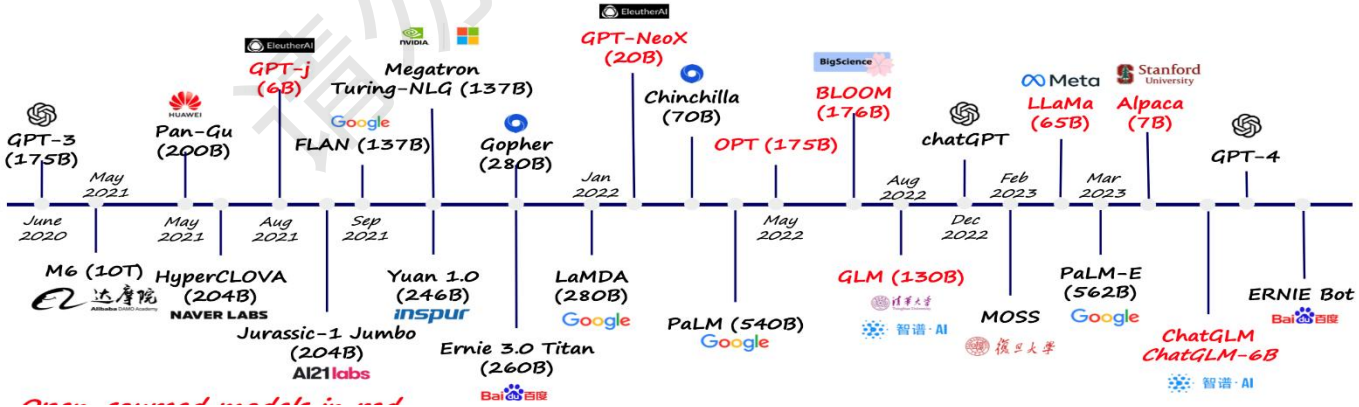


请勿私自上传本课件

1.1 ChatGPT与大模型简介

ChatGPT技术特点　AI大模型=大数据+大算力+终极算法





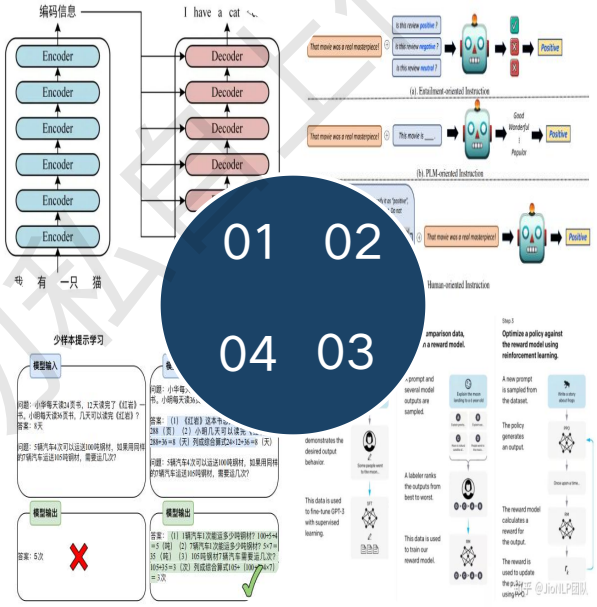
1.1 ChatGPT与大模型简介

ChatGPT技术特点

ChatGPT技术实现基础主要包括：Transformer 模型、激发出其所蕴含知识的提示学习与指令精调算法、思维链能力、以及确保其与人类意图对齐的基于人类反馈的强化学习算法。因嵌入了人类反馈强化学习以及监督微调，因而具备了理解上下文、保持连贯性等特征，在对话中能主动记忆先前的对话内容信息。

Transformer 预训练，巨大参数量

ChatGPT 基础模型采用 Transformer 架构。Transformer是一种基于自注意力机制的深度神经网络模型，可以高效并行地处理序列大数据，采用了数千亿 token 进行预训练。



提示学习与指令精调

提示学习（Prompt Learning）简单来说是通过编辑下游任务的输入，使其形式上模拟模型预训练过程使用的数据与任务。指令精调（InstructionTuning）通过构造“指令”并微调的方式，学习人类交互模式的分布，使模型更好的理解人类意图，与其对齐。

思维链提升推理能力

通过在小样本提示学习的示例中插入一系列中间推理步骤，有效提升了大模型的推理能力，模型可以将需要进行多步推理的问题分解为一系列的中间步骤，可以将计算资源分配到需要推理的问题上。

基于人工反馈的强化学习

ChatGPT使用监督微调训练一个初始模型。让AI trainer与chatbot进行聊天对话，创建强化学习的奖励模型。通过这些奖励模型，使用近端策略优化PPO算法对模型进行微调。

1.1 ChatGPT与大模型简介

■ 类ChatGPT通用大模型的利弊？

深度学习之父告别谷歌，称痛悔毕生工作，警告AI风险

2023-05-04 12:11 北京 来源：澎湃新闻·澎湃号·湃客 字号

原创 邵延港 猎云精选



“AI教父”陷入AI恐慌？

ChatGPT遭港大“封杀”：罪同论文剽窃抄袭，使用须报备过审

量子位QbitAI | + 关注

2023-02-20 11:43 北京 来源：澎湃新闻·澎湃号·湃客 字号

衡宇 发自 凹非寺

量子位 | 公众号 QbitAI

ChatGPT太能写了，论文、作业，哪里不会就帮写哪里。

为此，香港大学明文禁止师生将ChatGPT用于教学用途，违规使用将被判为抄袭。

港大也成为国内首家对ChatGPT进行明文规定的高校。

甘肃首例AI虚假信息案，男子用ChatGPT编假新闻牟利

元宇宙简史 | + 关注

2023-05-09 15:05 澳大利亚 来源：澎湃新闻·澎湃号·湃客 字号



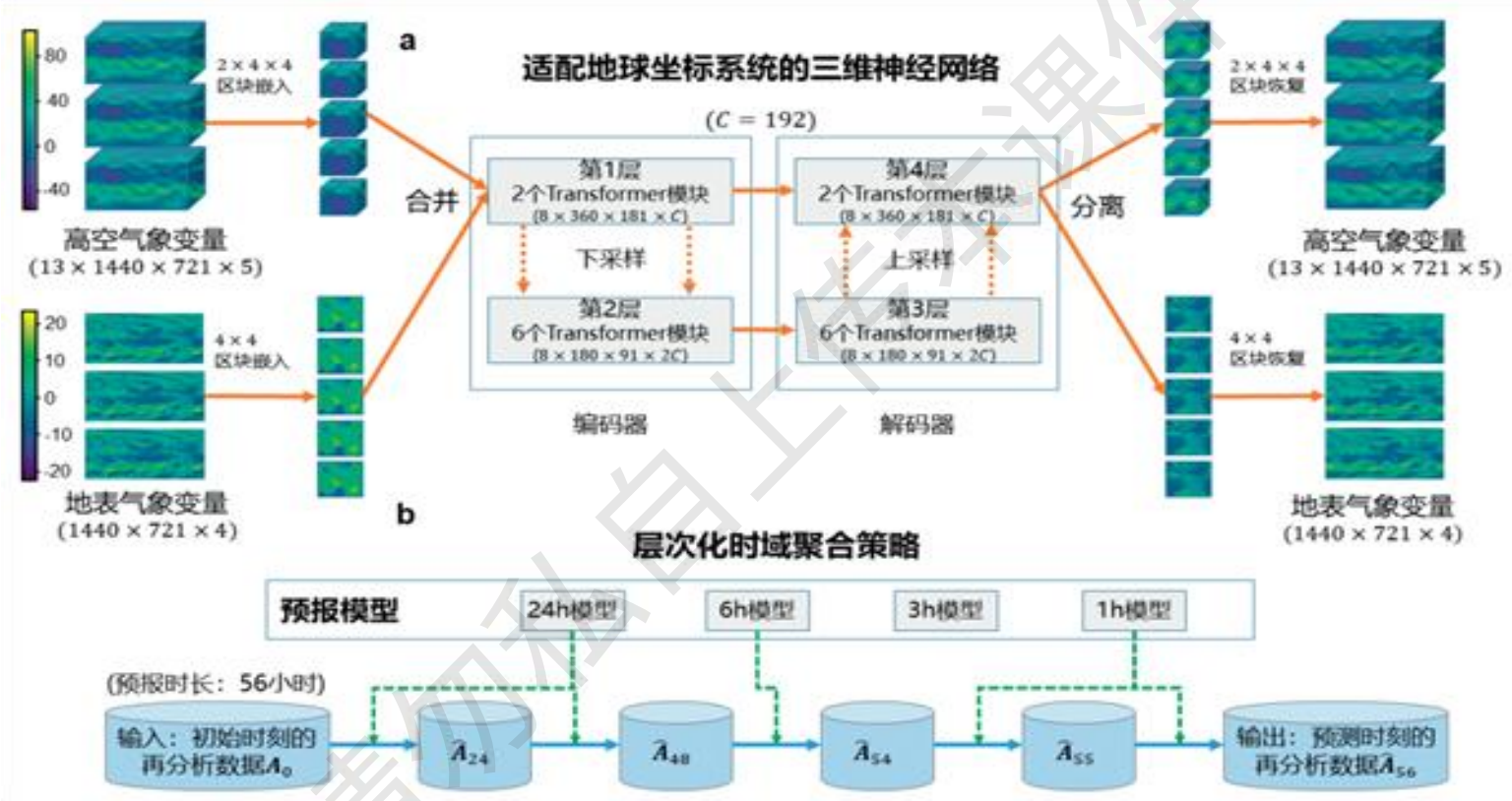
Hinton的灵魂四问：

- 我们是否应该让机器用宣传和谎言充斥我们的信息渠道？
- 我们是否应该把所有的工作都自动化，包括那些有成就感的工作？
- 我们是否应该开发非人类的大脑，淘汰我们并取代我们？
- 我们是否应该冒着失去对我们文明控制的风险？…

清华大学出版社

1.1 ChatGPT与大模型简介

■ 另一个大模型赛道 产业大模型-以华为盘古气象大模型为例



华为盘古气象大模型入选中国科学十大进展

Bi K, **e L, Zhang H, et al. Accurate medium-range global weather forecasting with 3D neural networks[J]. **Nature**, 2023, 619(7970): 533-538.

清华自上作系课件

1.1 ChatGPT与大模型简介

■ 另一个大模型赛道　产业大模型-华为技术方案



基础大模型》专业（行业）大模型》多场景多任务应用

请勿私自上传课件

生成式计算与大模型应用

1.1 ChatGPT与大模型简介

1.2 Transformer与大模型计算原理

1.3 Langchain与大模型开发框架*

请勿私自上传本课件

1.2 Transformer与大模型计算原理

■ 大模型核心技术　核心技术栈-五大方面

大模型技术栈

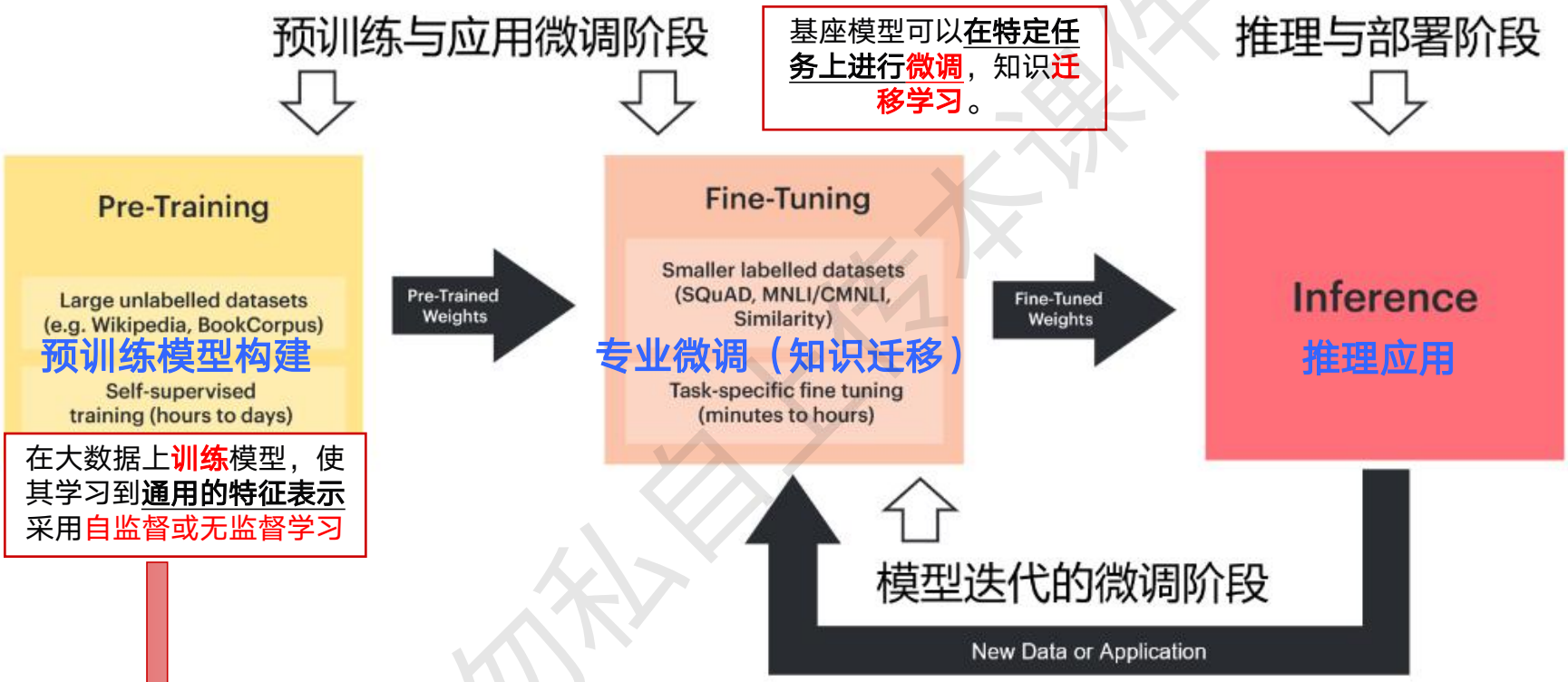


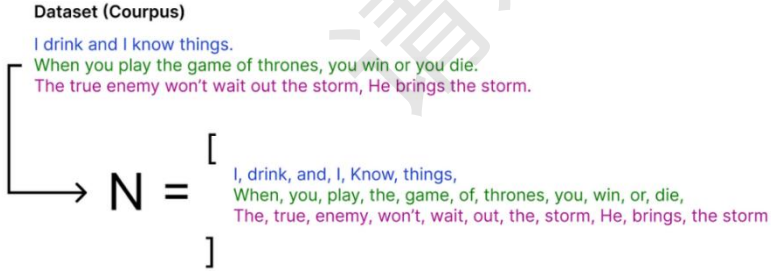
- **标记化(Tokenization)**：将文本序列转换为模型可以理解的单个单词、子词或标记的过程。
- **嵌入(Embedding)**：嵌入是单词或标记的连续**向量表示**，可捕获它们在高维空间中的语义。允许模型将离散标记转换为神经网络可以处理的格式。
- **注意力(Attention)**：计算给定上下文中不同单词或短语的**重要性**，对其分配不同的权重，使得模型可以专注最相关的信息。
- **预训练(Pretraining)**：在大型数据集上训练 LLM 的过程，通常是**无监督或自监督学习**一般语言模式、单词之间的关系以及其他基础知识，获得**基座大模型**。
- **微调**：利用预训练期间获得的知识（基座大模型）并将其应用于新的相关任务的技术，一般采用较小的特定任务数据集来**微调**预训练模型

请勿私自上传课件

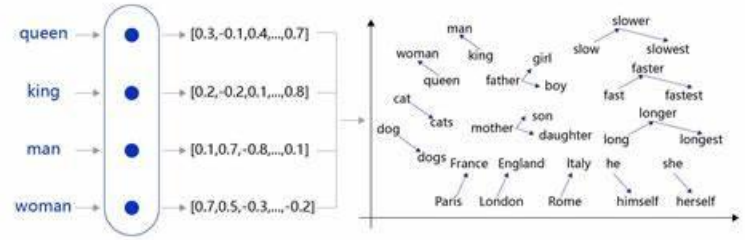
1.2 Transformer与大模型计算原理

■ 大模型核心技术　大模型应用流程





Tokenization(WordPiece, BPE等)



Embedding（Word2Vec）

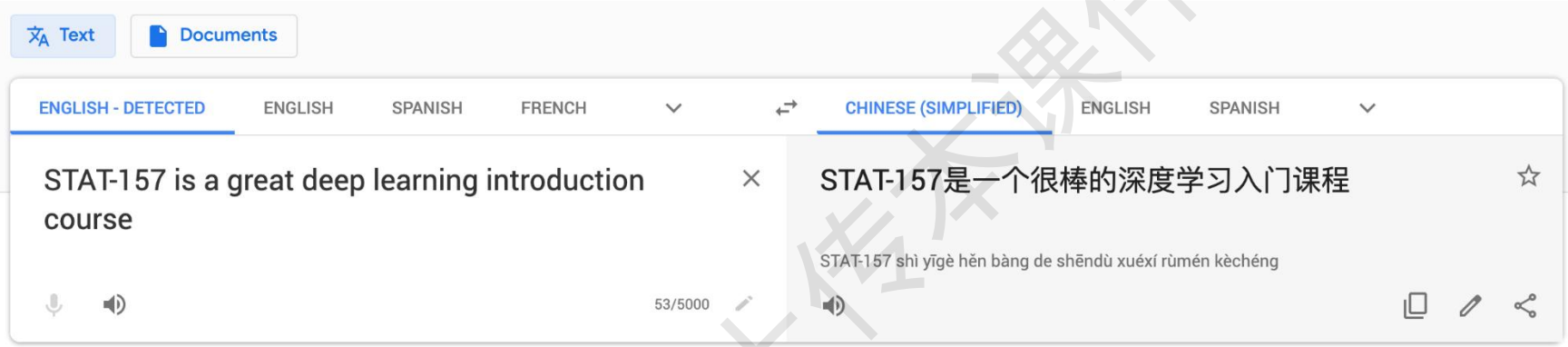
请勿私自上传本课件

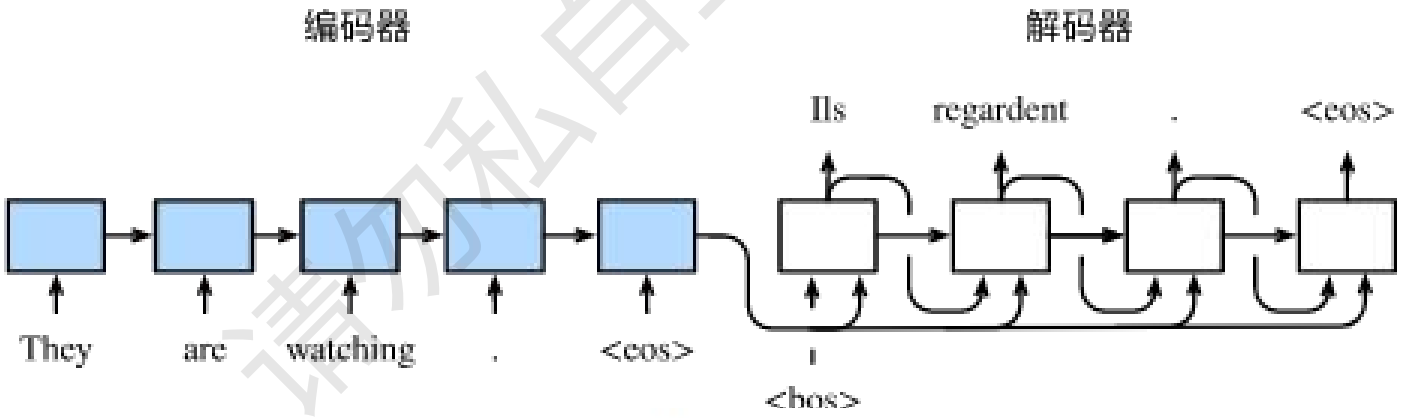
1.2 Transformer与大模型计算原理

- **大模型核心技术**

Transformer起源

基于经典的编码器-解码器结构





$$p_\theta(\mathbf{y}_{1:T}|\mathbf{x}_{1:S}) = \prod_{t=1}^{T} p_\theta(y_t|\mathbf{y}_{1:(t-1)}, \mathbf{x}_{1:S}),$$

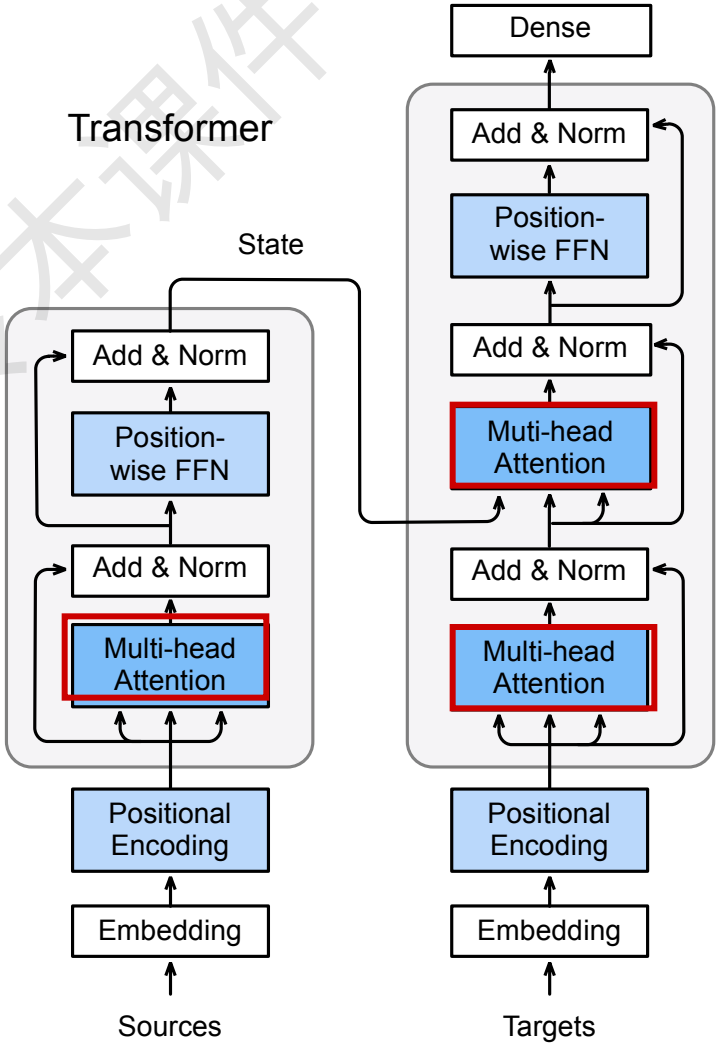
请勿私自上传本课件

1.2 Transformer与大模型计算原理

■ 大模型核心技术　　Transformer核心组件

Transformer 模型包含两个关键组件：编码器和解码器。编码器用于将输入序列映射到一组中间表示，解码器则将中间表示转换为目标序列。三个环节采用自注意力机制设计。

- 编码器 – 解码器架构
- 与经典编解码架构不同，注意力集中在 3 个地方
  - ✓ 位置编码
  - ✓ 自注意力
  - ✓ 层归一化
  - ✓ 直连边
  - ✓ 逐位的FNN

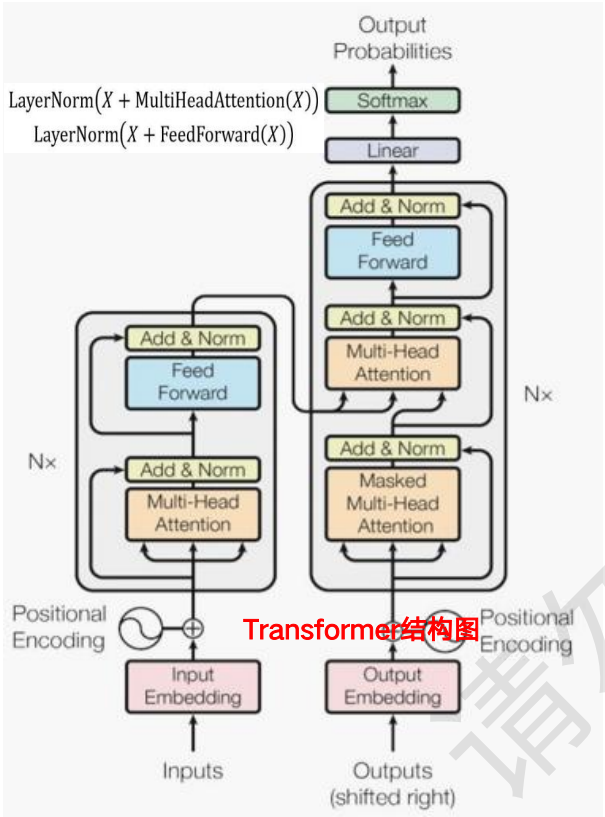


Transformer结构示意图

清华私自上传本课件

1.2 Transformer与大模型计算原理

■ 大模型核心技术

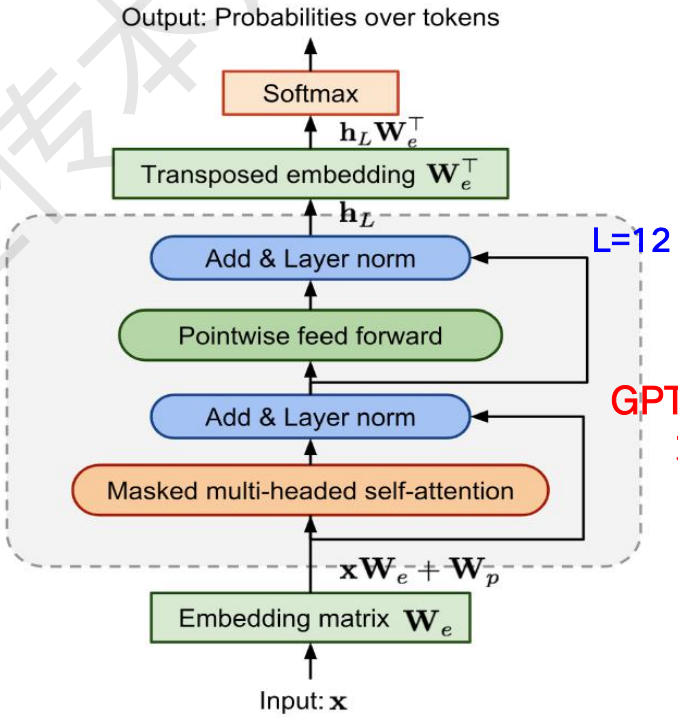


Transformer结构图

经典Transformer 结构**核心组件编码器和解码器**。

Transformer应用，以GPT-3为例

ChatGPT 的基础模型基于 Transformer 架构，是一种采用**自注意力机制的深度学习**模型，这一机制可以按输入数据各部分重要性的不同而计算不同的权重，可以高效并行地处理序列大数据。



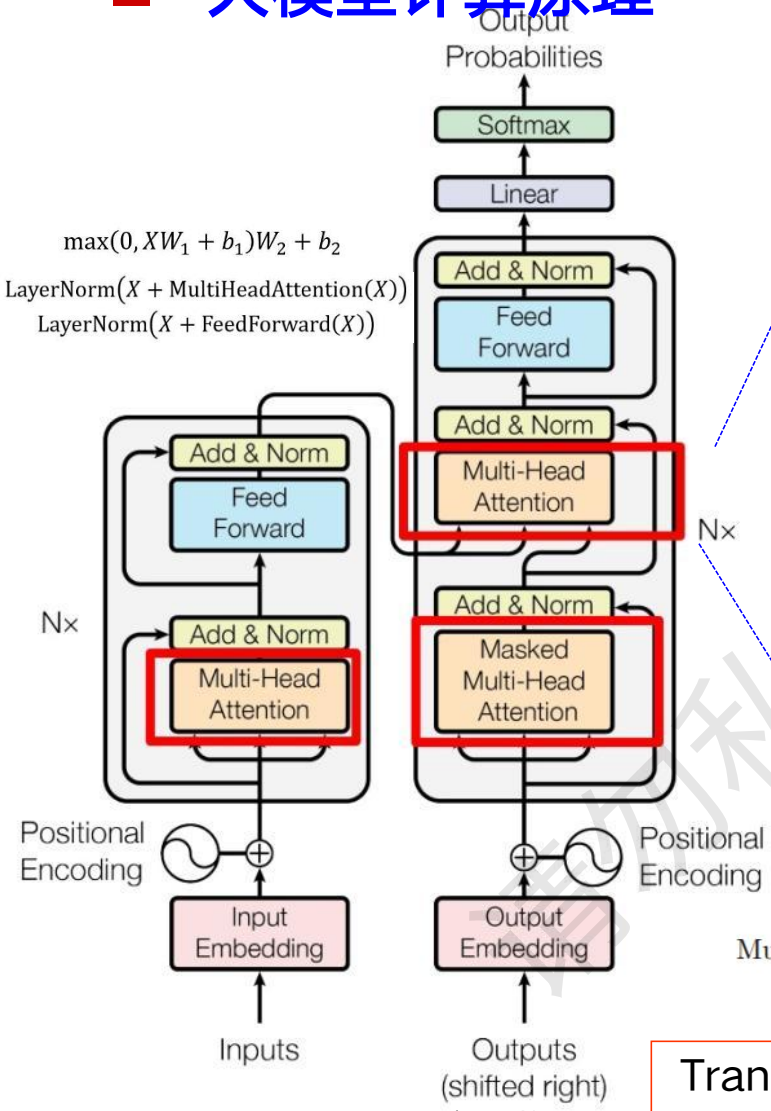
GPT-3 模型架构大道至简

GPT (Generative Pre-trained Transformer) 是由 OpenAI 提出的**只有解码器的预训练模型**。相较于之前的模型，不再需要对于每个任务采取不同的模型架构，而是首先预训练一个优异泛化能力的基座模型，再支持针对性地对下游任务进行微调，以**实现多任务通用**。

请勿私自上传本课件

1.2 Transformer与大模型计算原理

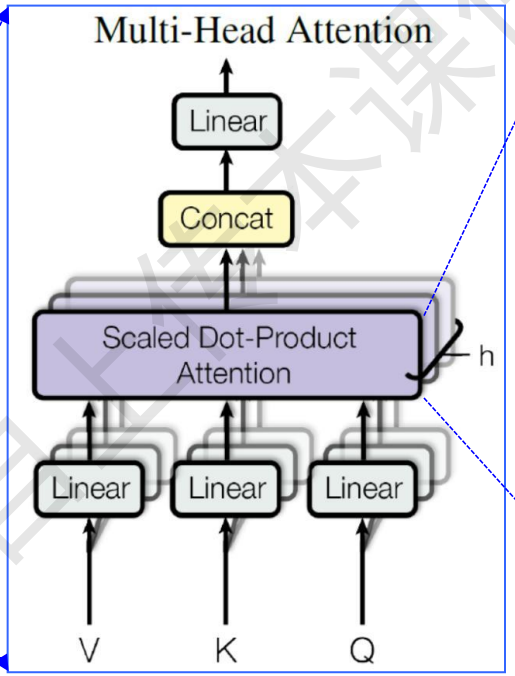
■ 大模型计算原理



Transformer主结构

大模型核心计算原理：Attention

多头注意力

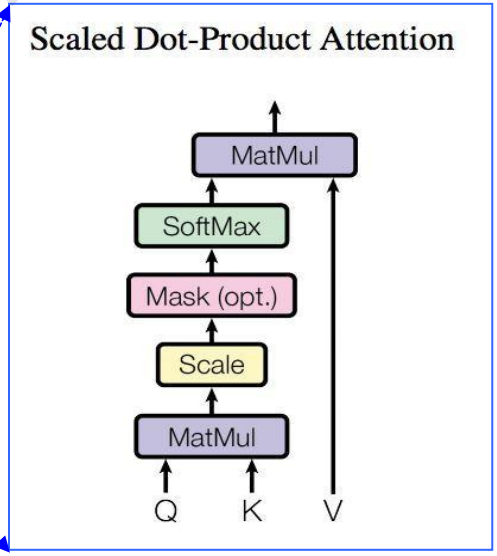


并行运行缩放点积注意力，独立的注意力输出矩阵通过Concat连接并线性转换为预期的维度

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_h)W^O$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

缩放点积注意力



计算查询向量Q与键向量K的点积，并经过softmax归一化得到注意力权重，再将这些权重应用于值向量V进行加权求和

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

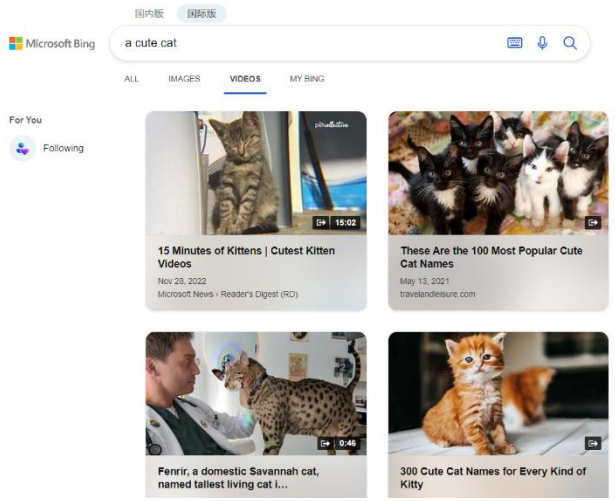
Transformer通过**QKV大矩阵计算注意力**，得到海量数据中的复杂**语义关联**度，并生成具有最高**概率**语义推理**输出**。

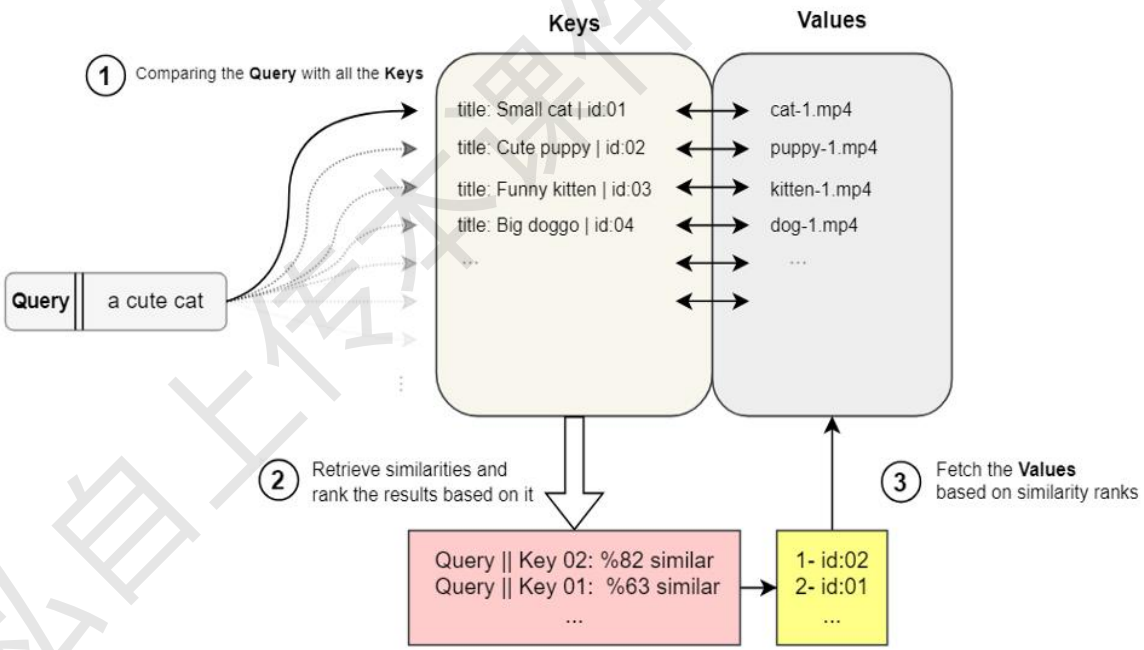
清风私自上传学术课件

1.2 Transformer与大模型计算原理

■ 大模型计算原理

注意力机制QKV计算的理解？





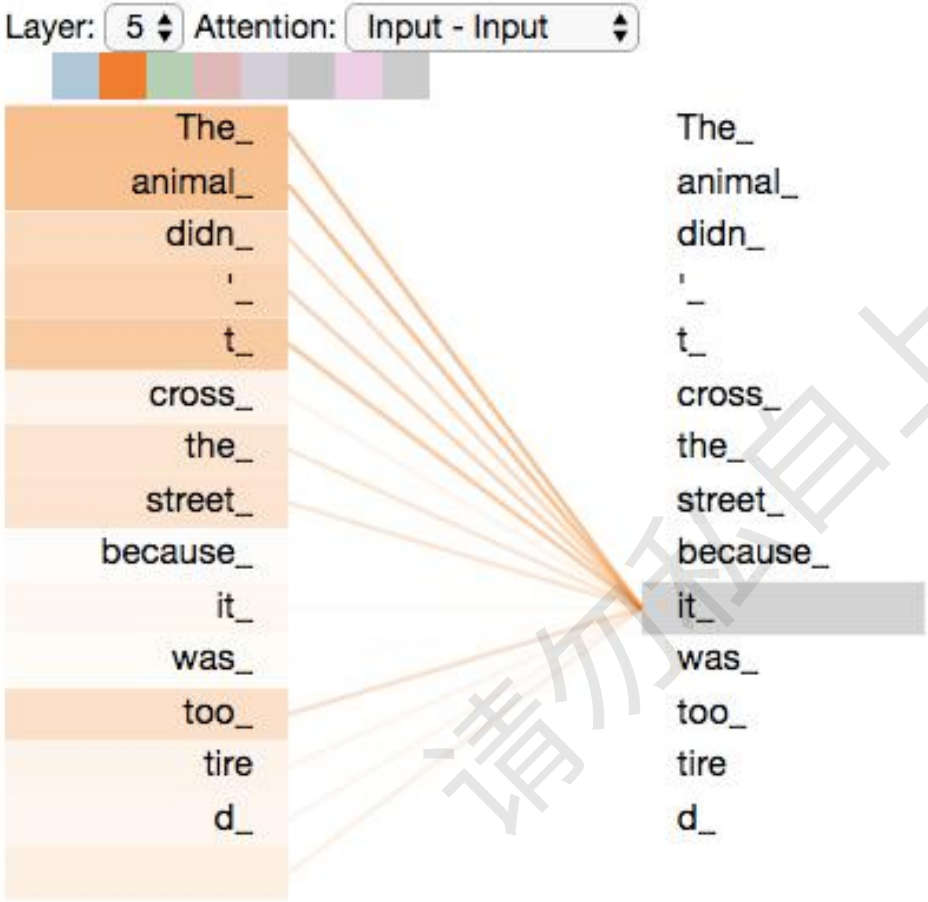
类似搜索引擎查询过程

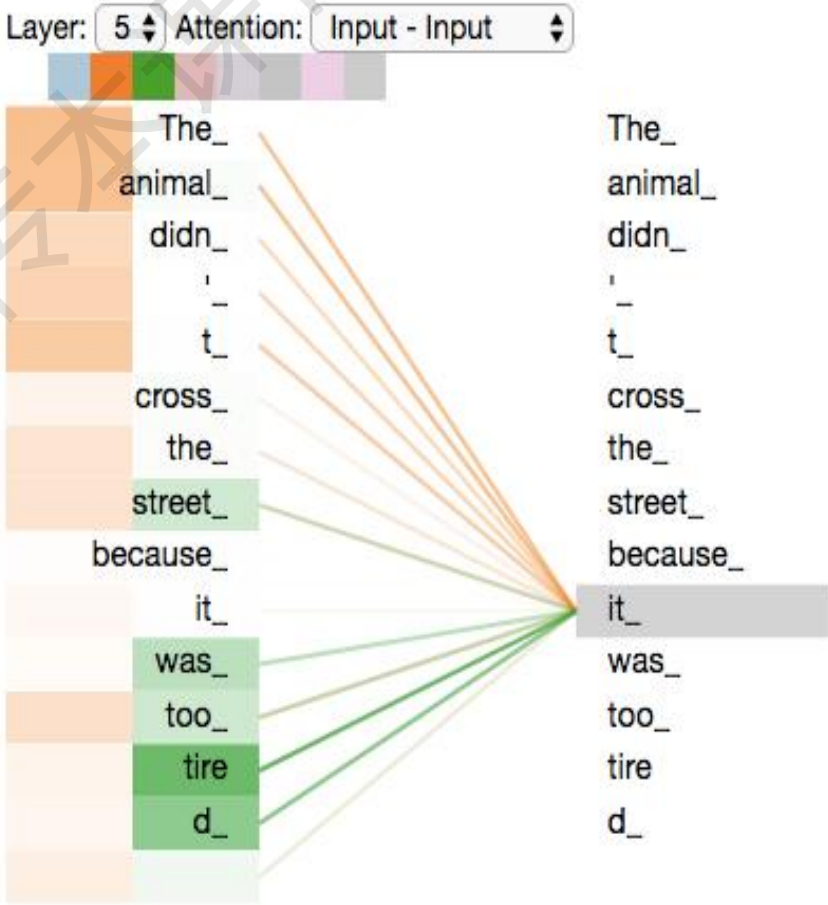
Q-query（对应搜索栏中输入的查询关键字）映射到数据库中与候选内容相关的一组关键字；
K-keys （对应搜索内容标题、关键词、描述等）匹配相关性，然后向你展示最匹配查询结果
V-values（最匹配的查找内容本身的链接）。
Attention 机制：将你想要查询的 query与搜索数据库中的keys 进行比较，并一对一地测量它们之间的相似度，最终从最高相似度顺序向下依次返回并排列索引到的查询结果内容values。

请勿私自上传未课件

1.2 Transformer与大模型计算原理

- **大模型计算原理** 注意力机制QKV计算的理解？



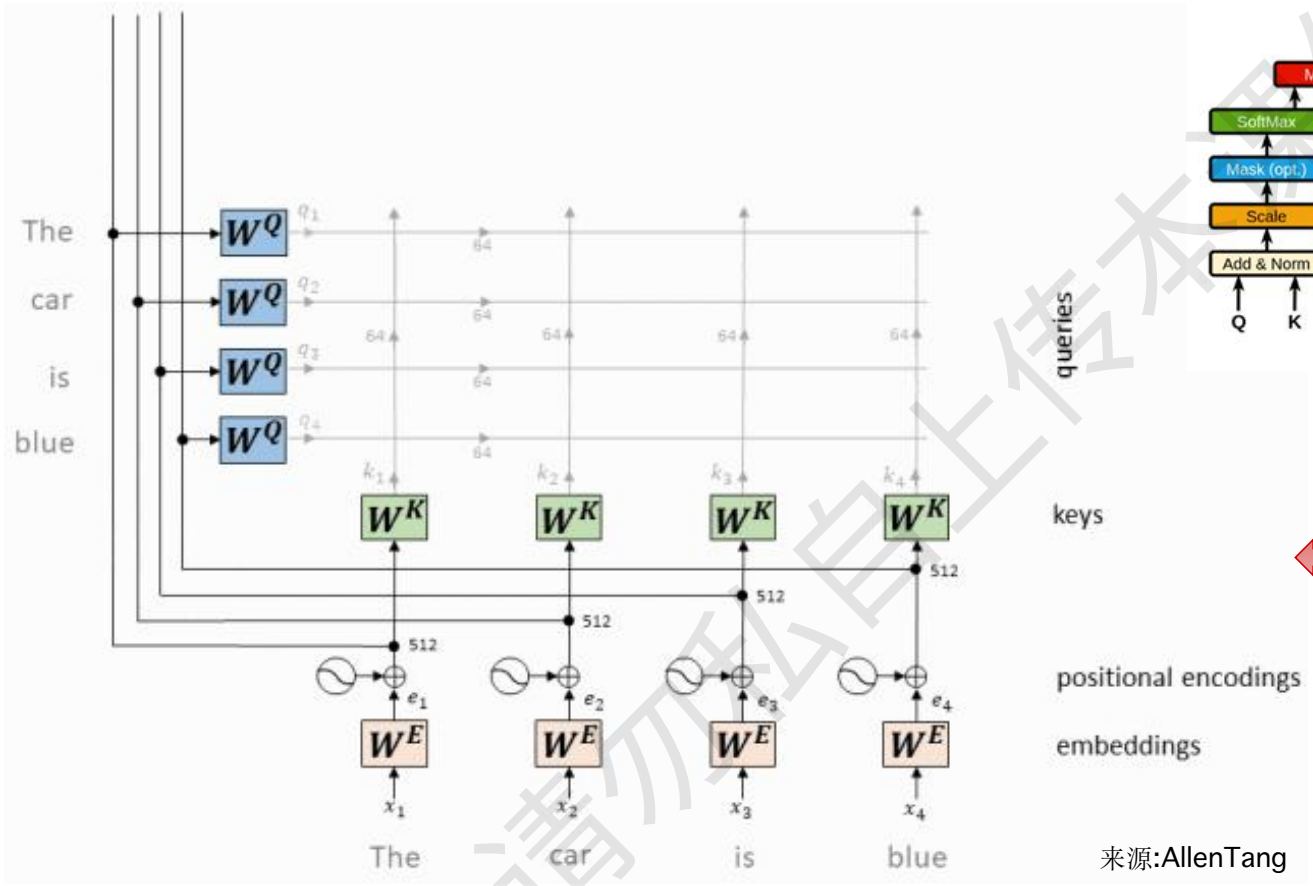


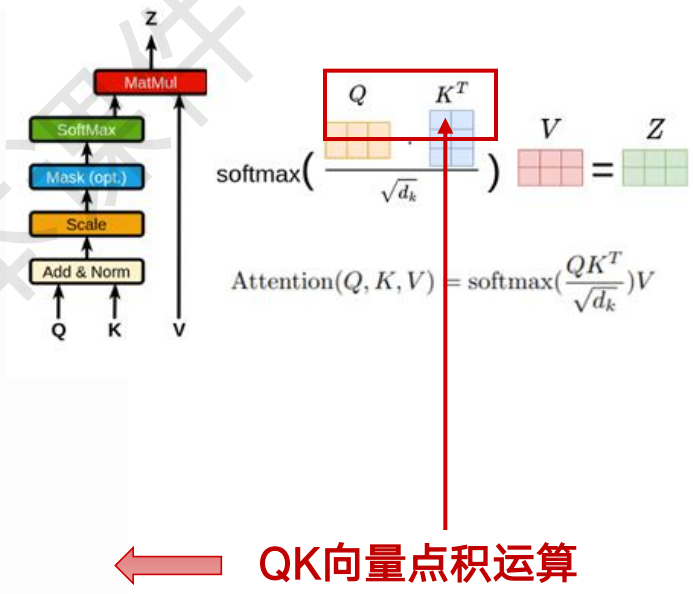
请勿私自上传未课件

1.2 Transformer与大模型计算原理

■ 大模型计算原理

注意力机制QKV计算过程的理解？





QK向量点积运算

来源:AllenTang

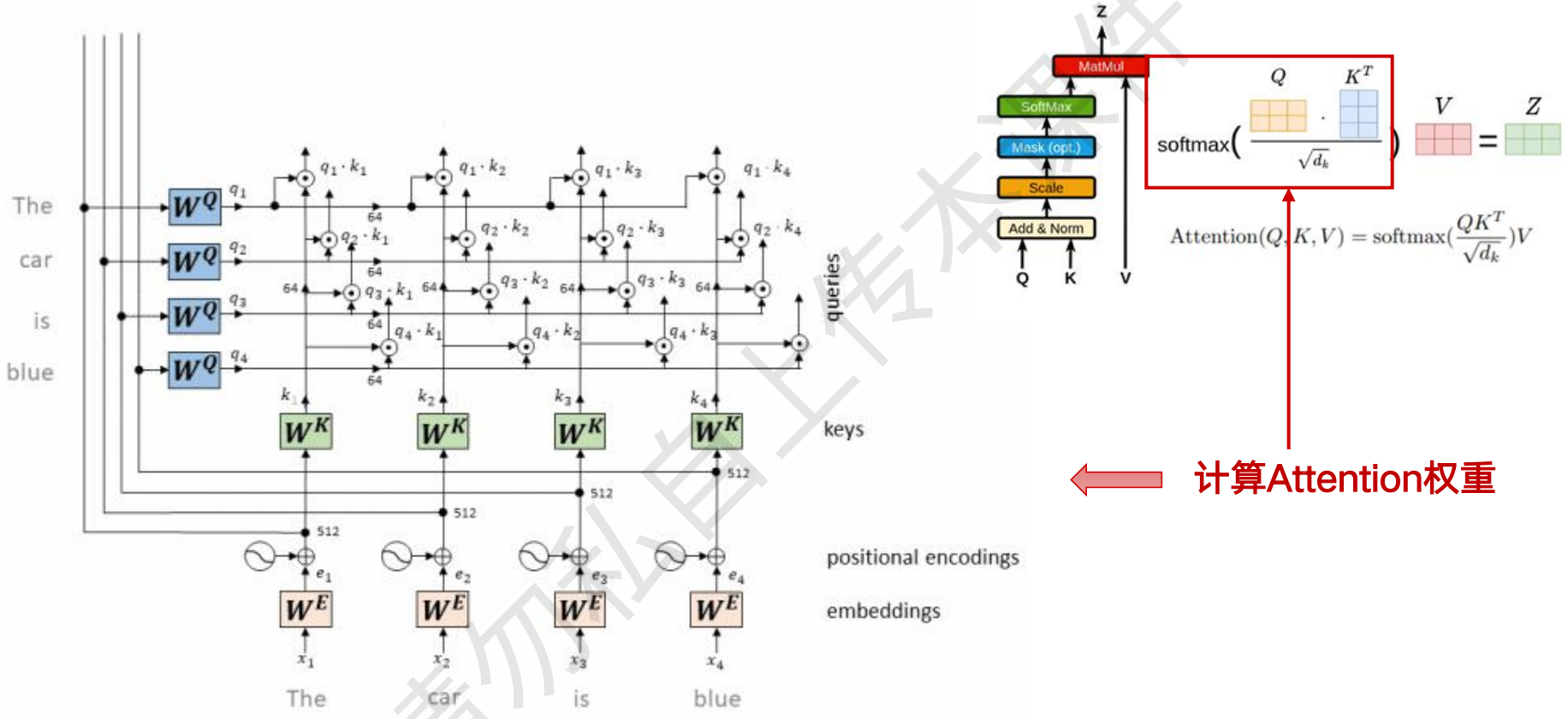
- 计算每个查询向量Q与所有键向量K的点积(对应元素相乘后求和);
- 这个步骤是在所有token之间建立关系，表示每个token对其他token的“关注”程度(计算得到Query与每个Key之间的相似度或匹配程度);

请勿私自上传本课件

1.2 Transformer与大模型计算原理

■ 大模型计算原理

注意力机制QKV计算过程的理解？

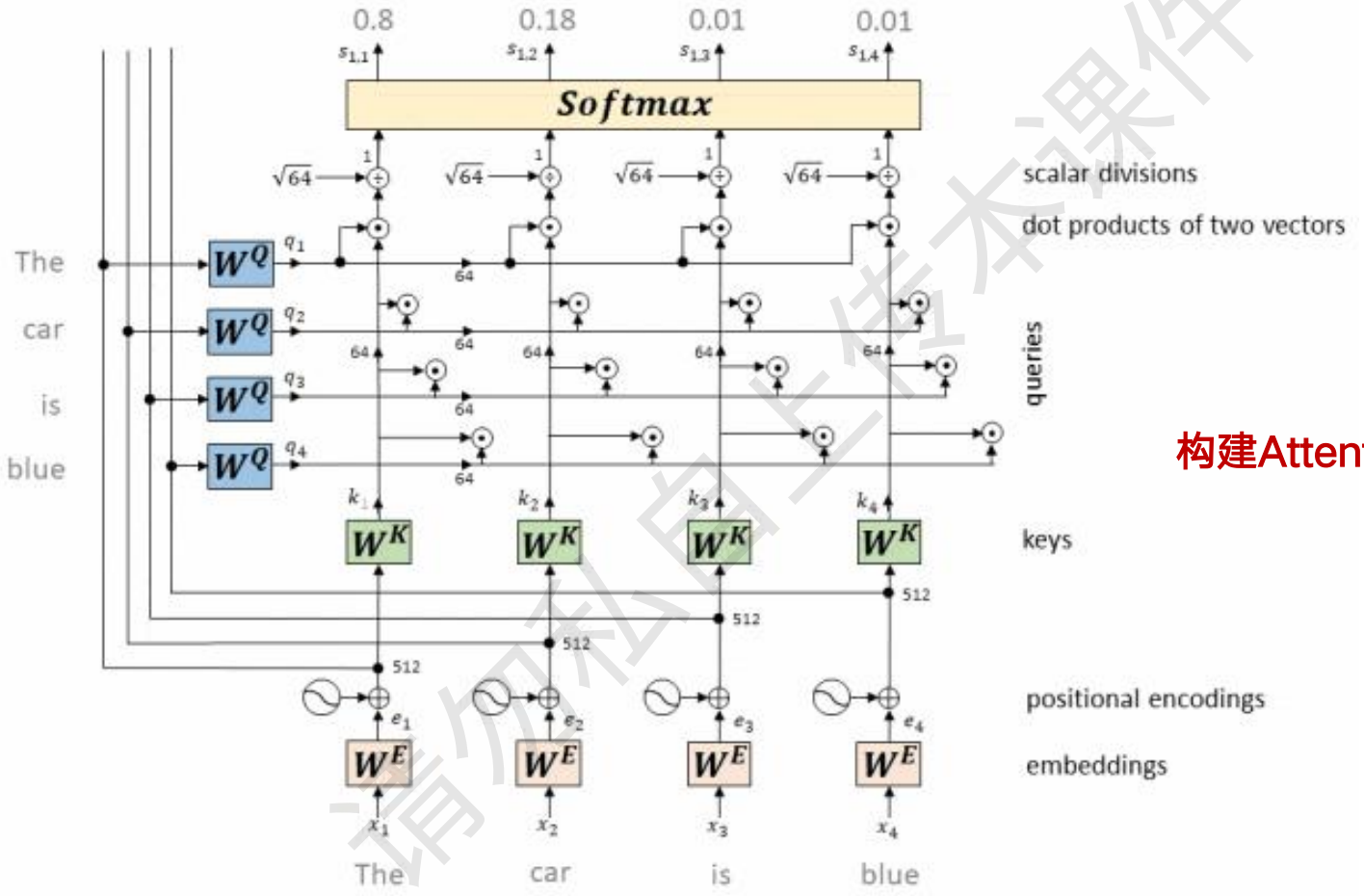


- 点积结果取softmax，得到Attention权重(0-1之间)；
- softmax函数将点积结果转换为0到1之间的值，即将原始得分转换为概率分布，表示每个token相对于其他所有token的Attention权重；
- 由于点积操作的结果可能非常大，因此引入缩放因子(输入维度dk的平方根)，让输入保持在一个合理的范围内；

1.2 Transformer与大模型计算原理

■ 大模型计算原理

注意力机制QKV计算过程的理解？



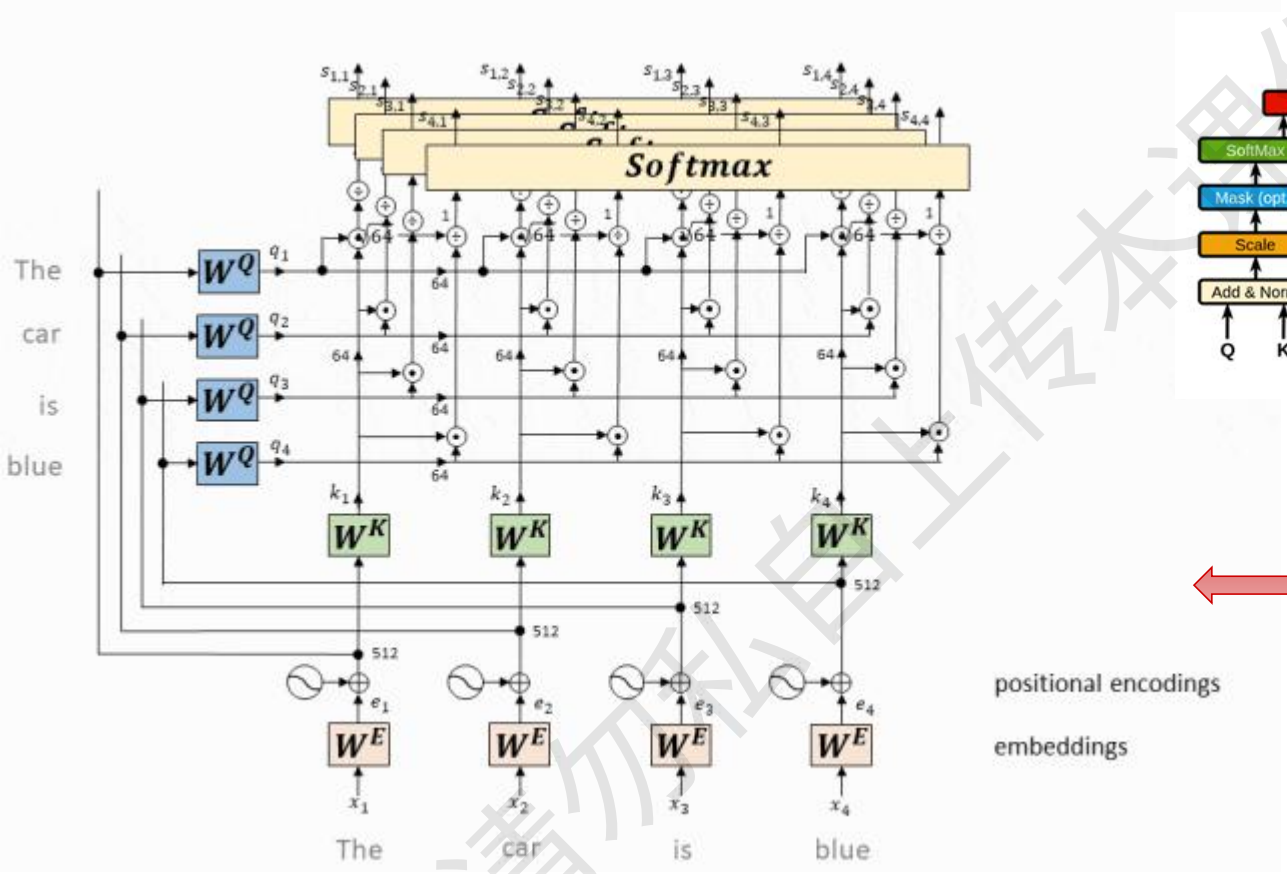
- 计算每个token相对于所有其它token的Attention权重（构成Attention矩阵）；
- 矩阵的每一行对应一个token，每一列也对应一个token，矩阵中的每个元素表示对应行token对列token的注意力权重;

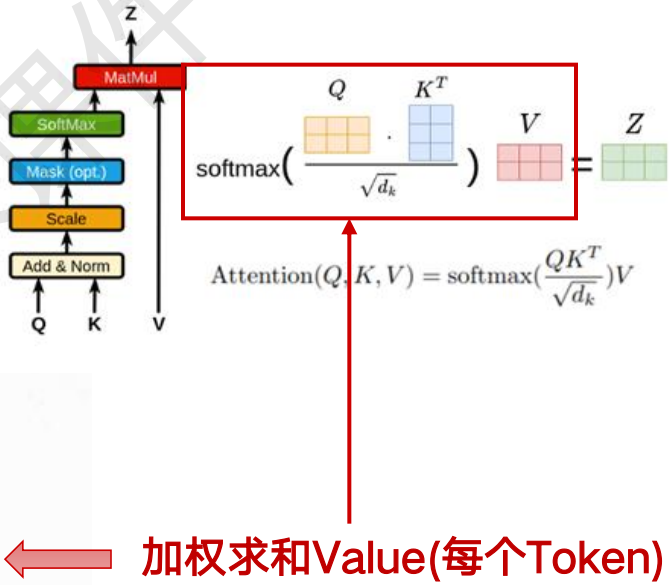
清华私自上传本课件

1.2 Transformer与大模型计算原理

■ 大模型计算原理

注意力机制QKV计算过程的理解？





加权求和Value(每个Token)

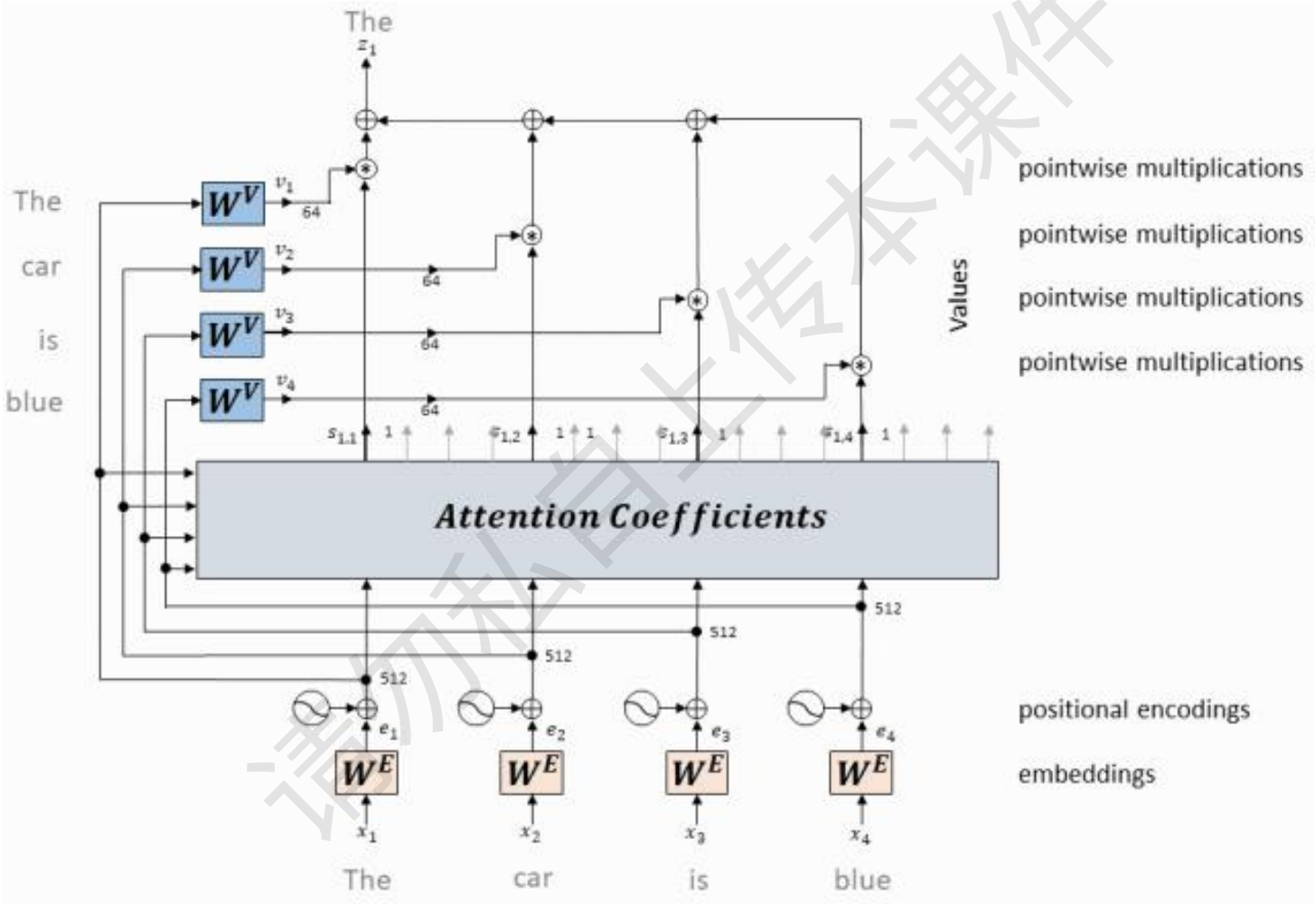
- 对于每个token，将其对应的值向量与Attention矩阵中该token所在行的所有权重相乘，并将结果相加;
- 这个加权求和的结果就是该token经过自注意力(Self-Attention)机制处理后的输出表示;

请勿私自上传本课件

1.2 Transformer与大模型计算原理

■ 大模型计算原理

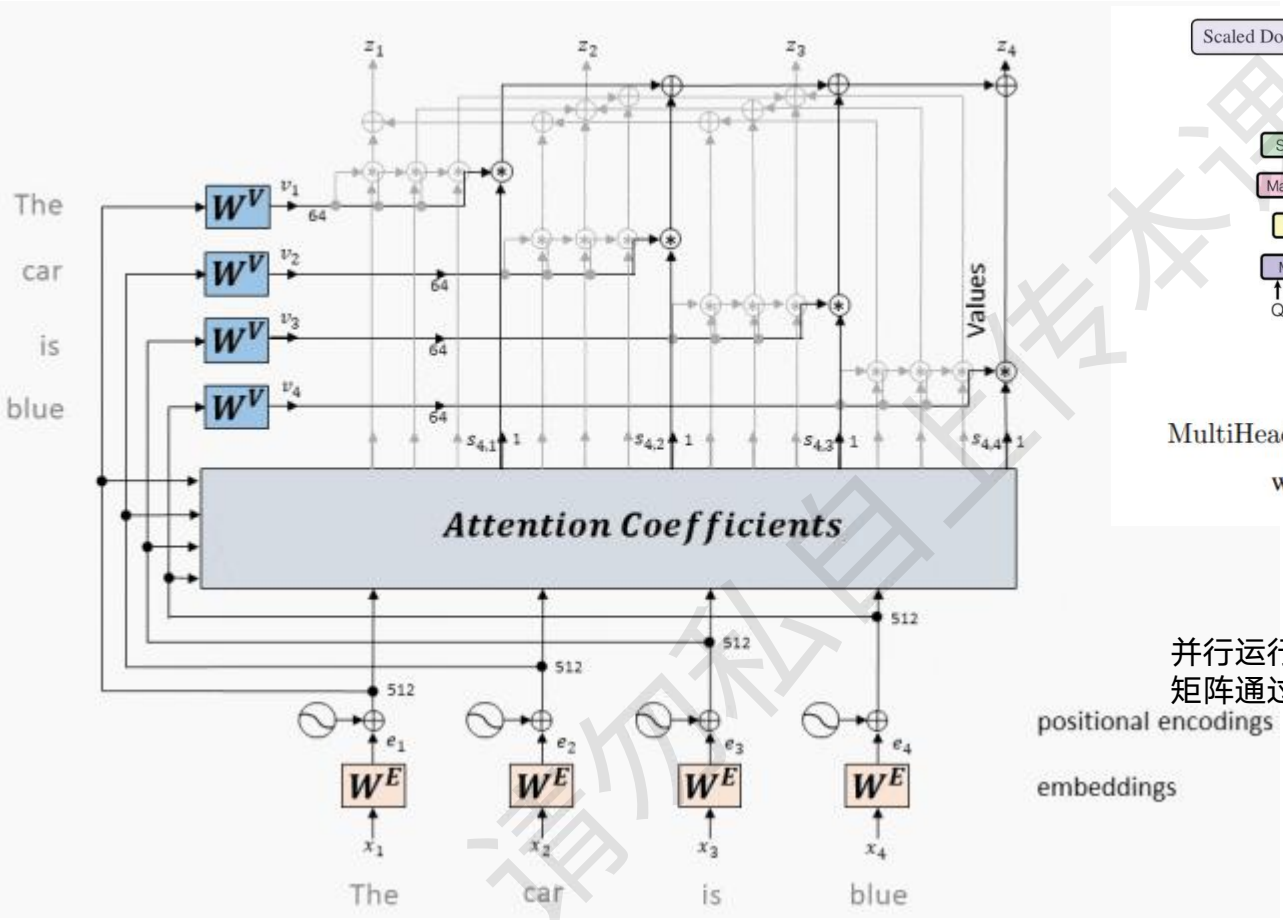
注意力机制QKV计算过程的理解？

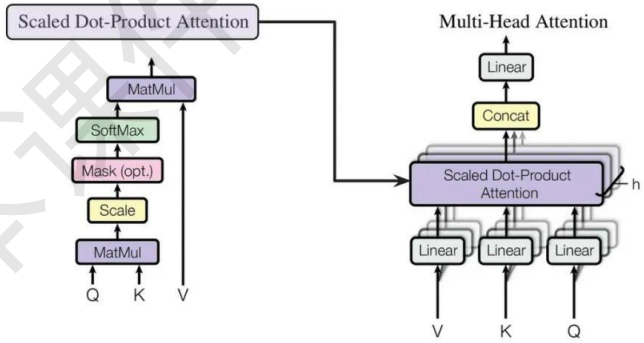


加权求和Value(所有Token)

1.2 Transformer与大模型计算原理

■ 大模型计算原理　　注意力机制QKV计算过程的理解？





$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1,...,\text{head}_h)W^O$$
$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

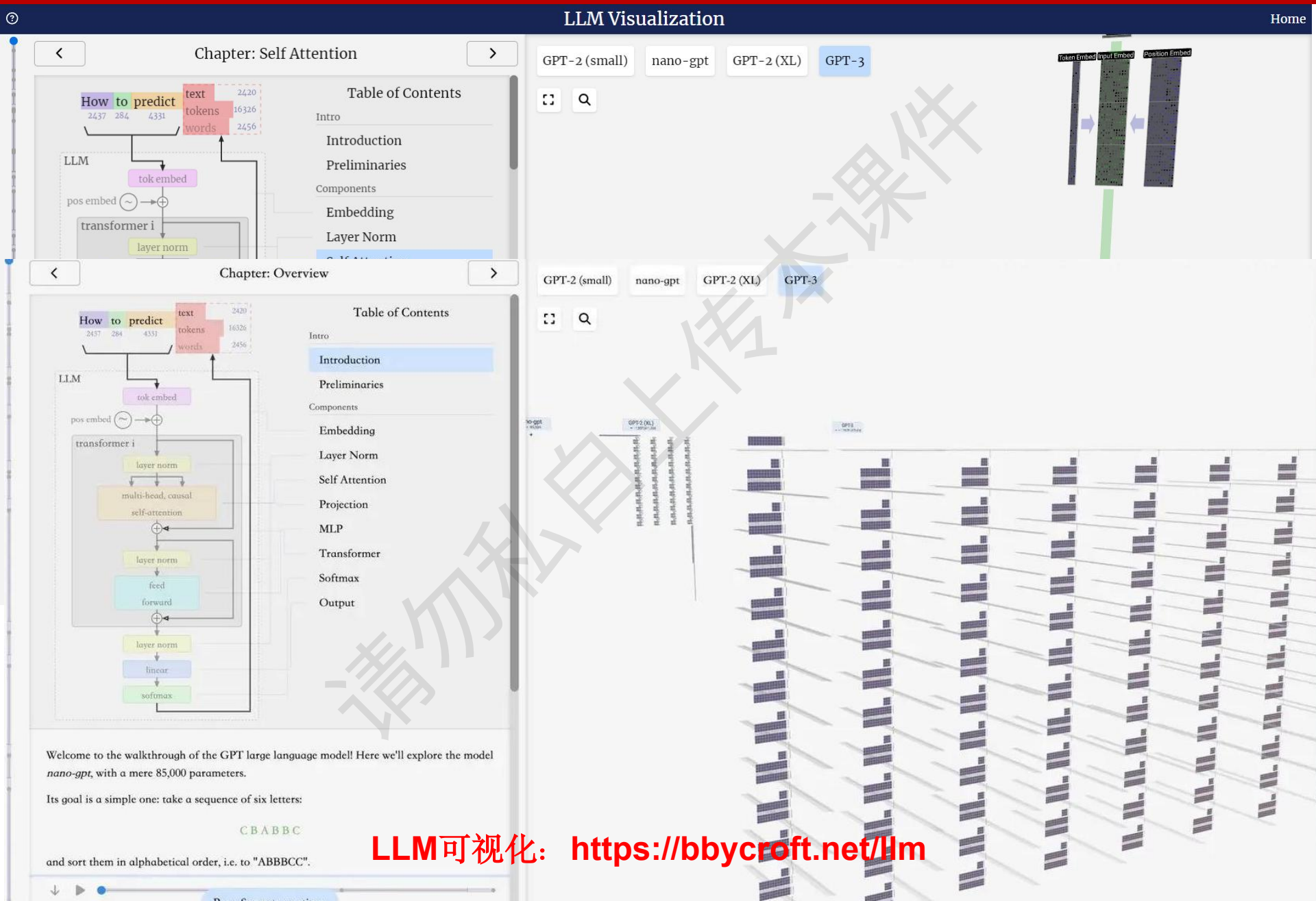
多头注意力拼接

并行运行缩放点积注意力，独立的注意力输出矩阵通过Concat连接并线性转换为预期的维度

- 对于输入的QKV向量，通过线性变换将它们映射到不同的子空间；
- 经过线性变换后，QKV向量被分割成多个头，每个头都会独立地进行前述自注意力计算；
- 将所有头的加权输出拼接在一起，然后通过一个线性变换得到最终的多头注意力输出。

请勿私自上传本课件

1.2 Transformer与大模型计算原理

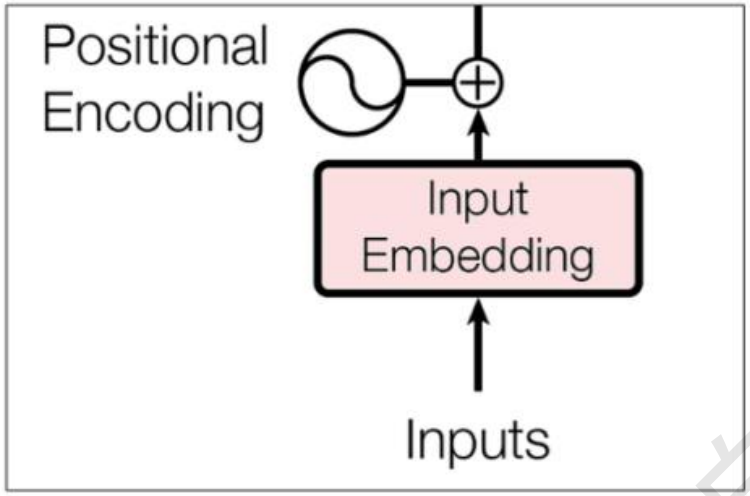


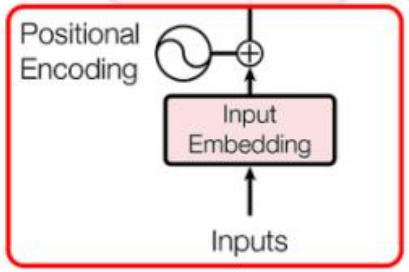
请勿私自上传本课件

LLM可视化：https://bbycroft.net/llm

Transformer计算过程实例

1、输入嵌入表示和位置编码





Step 1 (Defining the data)
Step 2 (Finding the Vocab Size)
Step 3 (Encoding and Embedding)
Step 4 (Positional Embedding)

请勿私自上传本课件

来源: Github: Best introduction to transformer, author:FareedKhan

Step 1 (Defining the data)
Step 2 (Finding the Vocab Size)
Step 3 (Encoding and Embedding)
Step 4 (Positional Embedding)

Dataset (Courpus)

1. I drink and I know things.
2. When you play the game of thrones, you win or you die.
3. The true enemy won't wait out the storm, He brings the storm.

获得数据集

Dataset (Courpus)

I drink and I know things.
When you play the game of thrones, you win or you die.
The true enemy won't wait out the storm, He brings the storm.

→ N = [
I, drink, and, I, Know, things,
When, you, play, the, game, of, thrones, you, win, or, die,
The, true, enemy, won't, wait, out, the, storm, He, brings, the storm
]

数据集分解为一个token列表

$$vocab\ size = count(set(N))$$

→ set (I, drink, and, I, Know,
When, you, play, the, game, of, thrones, you, win, or, die
The, true, enemy, won't, wait, out, the, storm, He, brings, the storm)

→ count (I, drink, and, Know, When, you, play, the, game, of, thrones,
win, or, die, true, enemy, won't, wait, out, storm, He, brings,
storm)

→ = 23

计算词汇量

请勿私自上传本课件

Step 1 (Defining the data)
Step 2 (Finding the Vocab Size)
Step 3 (Encoding and Embedding)
Step 4 (Positional Embedding)

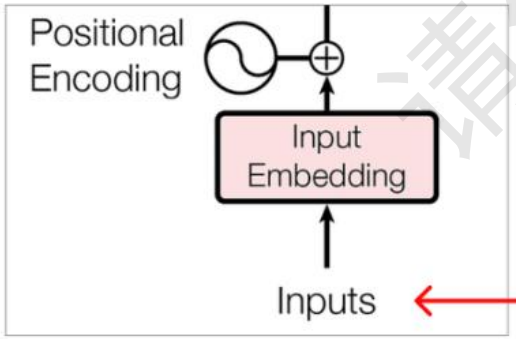
每个唯一单词分配一个整数作为编号

1	2	3	4	5	6	7	8	9	10	11	12
I	drink	storm	Know	When	won't	play	out	true	storm	brings	game

13	14	15	16	17	18	19	20	21	22	23
the	win	of	enemy	you	wait	thrones	and	or	die	He

1	2	3	4	5	6	7	8	9	10	11	12
I	drink	storm	Know	When	won't	play	out	true	storm	brings	game

13	14	15	16	17	18	19	20	21	22	23
the	win	of	enemy	you	wait	thrones	and	or	die	He

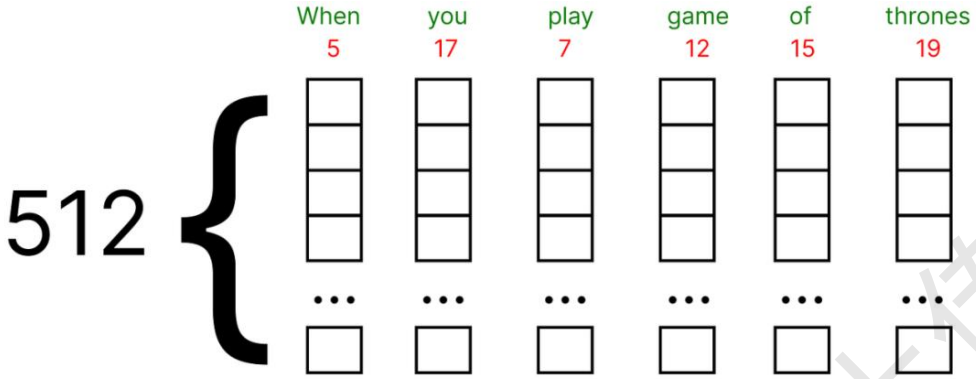


When	you	play	game	of	thrones
5	17	7	12	15	19

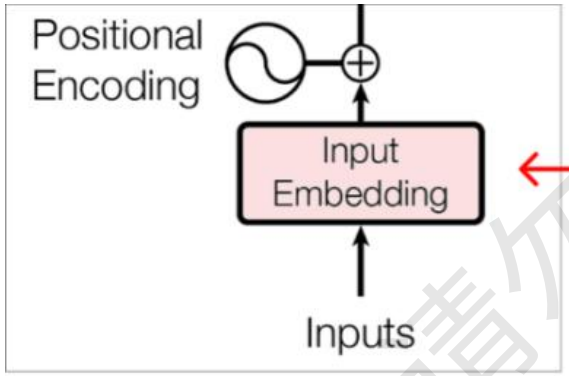
在对整个数据集进行编码之后，从语料库中选择一个句子开始

请勿私自上传本课件

Step 1 (Defining the data)
Step 2 (Finding the Vocab Size)
Step 3 (Encoding and Embedding)
Step 4 (Positional Embedding)



输入的每个token将被表示为一个编码，embedding可以通过Word2vec计算得到



When	you	play	game	of	thrones
5	17	7	12	15	19
e1	e2	e3	e4	e5	e6
0.19	0.07	0.94	0.43	0.8	0.34
0.6	0.7	0.73	0.49	0.65	0.72
0.45	0.96	0.02	0.09	0.57	0.79
0.56	0.97	0.85	0.21	0.84	0.75
0.13	0.58	0.16	0.26	0.05	0.75

原论文Embedding向量维度512，本示例设为5。

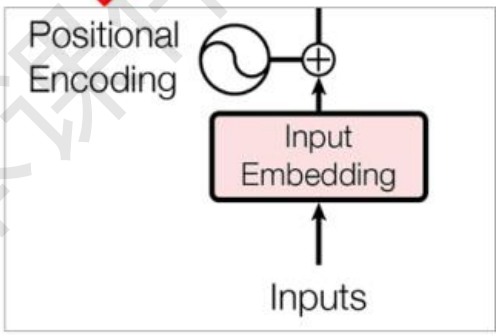
请勿私自上传本课件

Step 1 (Defining the data)
Step 2 (Finding the Vocab Size)
Step 3 (Encoding and Embedding)
Step 4 (Positional Embedding)

第一个单词 “when”的POS值将为零，因为它对应于序列的起始索引。此外，i的值 (取决于是偶数还是奇数) 决定了用于计算PE值的公式。

For Single word Calculation of positional Embedding

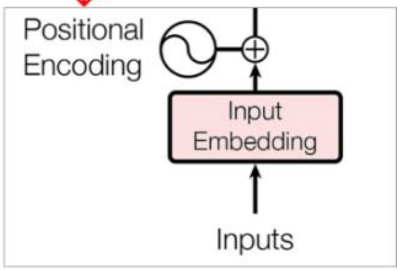
	When			
	5			
	e1			p1
i = 0	0.19	$PE(Even) = cos(pos/10000^{2i/dim})$	$cos(0/10000^{(2*0)/5})$	1
i = 1	0.6	$PE(Odd) = sin(pos/10000^{2i/dim})$	$sin(0/10000^{(2*1)/5})$	0
i = 2	0.45	$PE(Even) = cos(pos/10000^{2i/dim})$	$cos(0/10000^{(2*2)/5})$	1
i = 3	0.56	$PE(Odd) = sin(pos/10000^{2i/dim})$	$sin(0/10000^{(2*3)/5})$	0
i = 4	0.13	$PE(Even) = cos(pos/10000^{2i/dim})$	$cos(0/10000^{(2*4)/5})$	1
	dim (d) = 5			
	pos = 0			



$$PE(Odd) = sin(pos/10000^{2i/dim})$$
$$PE(Even) = cos(pos/10000^{2i/dim})$$

fo all words in our first input, finding positional embedding

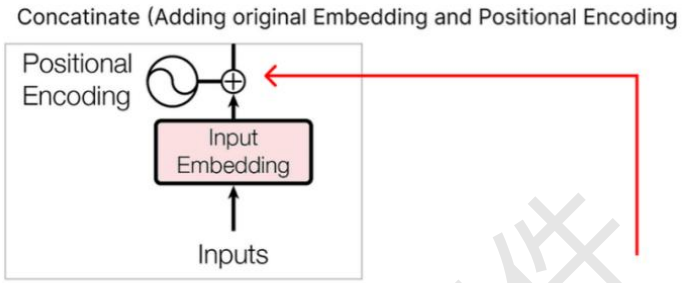
	When	you	play	game	of	thrones
	5	17	7	12	15	19
	p1	p2	p3	p4	p5	p6
	1	0.540302	-0.41615	-0.989992497	-0.65364362	0.283662185
	0	0.025116	0.050217	0.075285293	0.100306487	0.125264396
	1	1	0.999999	0.999998209	0.999996815	0.999995024
	0	1.58E-05	3.17E-05	4.75468E-05	6.33957E-05	7.92447E-05
	1	1	1	1	1	1
POS =	0	1	2	3	4	5
dim (d) =	5	5	5	5	5	5



继续计算位置embedding，为下一个单词“you”分配pos值1，并继续为序列中的每个后续单词递增pos值。

请勿私自上传未课件

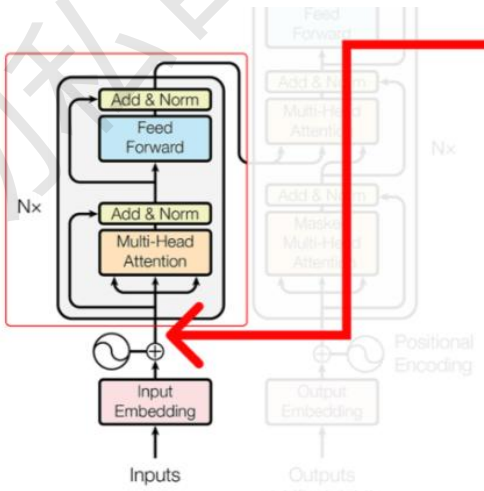
Step 1 (Defining the data)
Step 2 (Finding the Vocab Size)
Step 3 (Encoding and Embeddin
Step 4 (Positional Embedding)



找到位置embedding后，将其与原始单词embedding联系起来。

When	you	play	game	of	thrones
5	17	7	12	15	19
e1	e2	e3	e4	e5	e6
0.19	0.07	0.94	0.43	0.8	0.34
0.6	0.7	0.73	0.49	0.65	0.72
0.45	0.96	0.02	0.09	0.57	0.79
0.56	0.97	0.85	0.21	0.84	0.75
0.13	0.58	0.16	0.26	0.05	0.75
+	+	+	+	+	+
p1	p2	p3	p4	p5	p6
1	0.540302306	-0.416146837	-0.989992497	-0.653643621	0.283662185
0	0.025116223	0.050216599	0.075285293	0.100306487	0.125264396
1	0.999999801	0.999999204	0.999998209	0.999996815	0.999995024
0	1.58489E-05	3.16979E-05	4.75468E-05	6.33957E-05	7.92447E-05
1	1	1	1	1	1

得到的结果向量是e1+p1，e2+p2，e3+p3等mbedding和

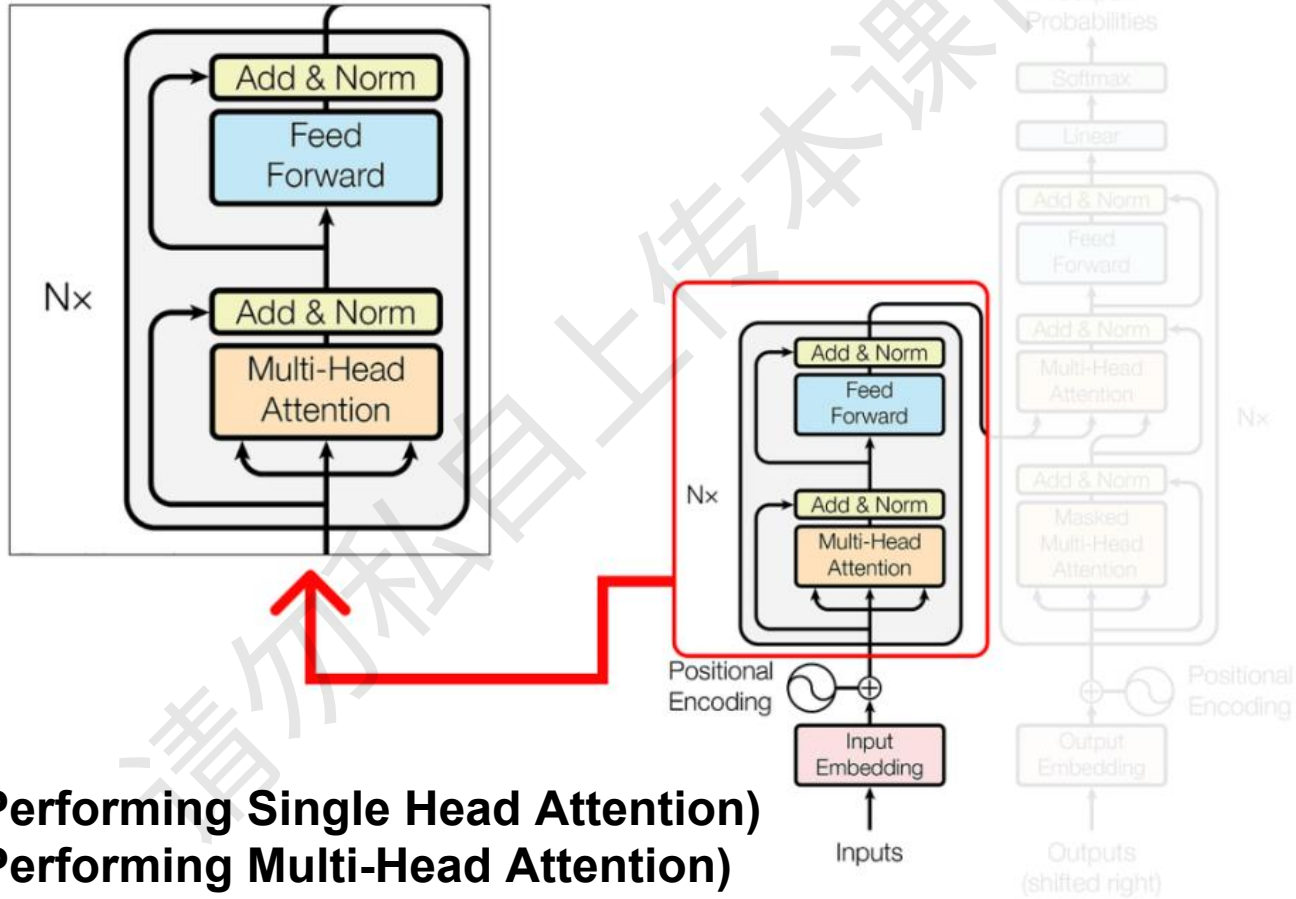


When	you	play	game	of	thrones
5	17	7	12	15	19
ep1	ep2	ep3	ep4	ep5	ep6
1.19	0.6103	0.5239	-0.56	0.1464	0.6237
0.6	0.7251	0.7802	0.5653	0.7503	0.8453
1.45	1.96	1.02	1.09	1.57	1.79
0.56	0.97	0.85	0.21	0.8401	0.7501
1.13	1.58	1.16	1.26	1.05	1.75

请勿私自上传非课件

Transformer计算过程实例

2、编码器计算

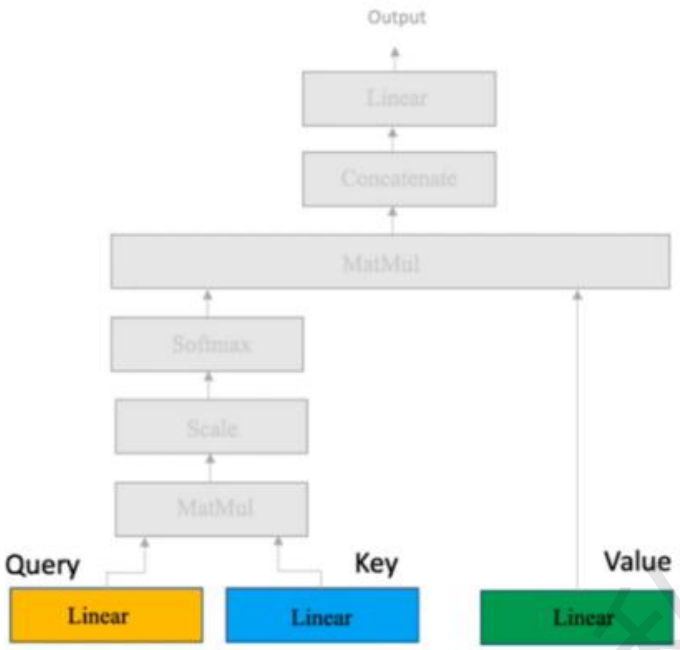


Step 1 (Performing Single Head Attention)
Step 2 (Performing Multi-Head Attention)

请勿私自上传本课件

Step 1 (Performing Single Head Attention)

Step 2 (Performing Multi-Head Attention)





三个输入矩阵表示从将位置embedding添加到单词embedding矩阵的上一步获得的转置输出。线性权重矩阵（黄色，蓝色和红色）表示注意力机制中使用的权重。这些矩阵的列可以具有任意数量的维数，但是行数必须与用于乘法的输入矩阵中的列数相同。本示例假设线性矩阵（黄色，蓝色和红色）包含随机权重，然后在训练过程中通过反向传播和梯度下降进行调整。

请勿私自上传本课件

Step 1 (Performing Single Head Attention)

Step 2 (Performing Multi-Head Attention)

	Embedding + Positional						Linear weights				Query		
when	1.19	0.6	1.5	0.56	1.13		0.552	0.26	0.208		2.8	2.3	2.44
you	0.61	0.73	2	0.97	1.58		0.885	0.21	0.262		3.3	3	3.33
play	0.52	0.78	1	0.85	1.16	X	0.178	0.89	0.297	=	2.7	2.1	2.54
game	-0.56	0.57	1.1	0.21	1.26		0.605	0.9	0.994		1.7	1.2	1.74
of	0.15	0.75	1.6	0.84	1.05		0.906	0.07	0.931		2.5	2.4	2.51
thrones	0.62	0.85	1.8	0.75	1.75						3.5	2.7	3.26

	Embedding + Positional						Linear weights				Key		
when	1.19	0.6	1.5	0.56	1.13		0.996	0.22	0.237		2.9	1.6	1.9
you	0.61	0.73	2	0.97	1.58		0.667	0.87	0.785		3	2	2.45
play	0.52	0.78	1	0.85	1.16	X	0.587	0.1	0.15	=	2.3	1.6	2.06
game	-0.56	0.57	1.1	0.21	1.26		0.565	0.17	0.793		0.7	1.2	1.18
of	0.15	0.75	1.6	0.84	1.05		0.122	0.52	0.427		2.2	1.5	1.97
thrones	0.62	0.85	1.8	0.75	1.75						2.9	2.1	2.42

	Embedding + Positional						Linear weights				Value		
when	1.19	0.6	1.5	0.56	1.13		0.896	0.58	0.019		3	2.8	1.98
you	0.61	0.73	2	0.97	1.58		0.045	0.43	0.594		3.3	3.3	2.7
play	0.52	0.78	1	0.85	1.16	X	0.717	0.52	0.495	=	2.3	2.4	1.97
game	-0.56	0.57	1.1	0.21	1.26		0.658	0.27	0.288		1	1.6	1.73
of	0.15	0.75	1.6	0.84	1.05		0.443	0.87	0.64		2.3	2.4	2.14
thrones	0.62	0.85	1.8	0.75	1.75						3.2	3.4	2.74

计算QKV矩阵

Step 1 (Performing Single Head Attention)

Step 2 (Performing Multi-Head Attention)

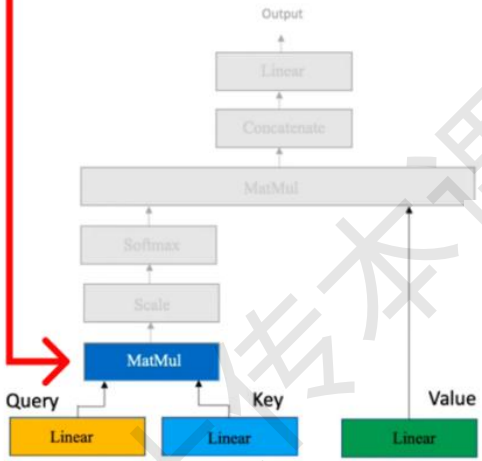
$$query * transpose(key)$$

Query		
2.8	2.31	2.44
3.35	3.04	3.33
2.73	2.05	2.54
1.65	1.22	1.74
2.48	2.42	2.51
3.45	2.73	3.26

X

Transpose(Key)					
2.89	2.98	2.26	0.731047	2.170039	2.872435
1.62	1.95	1.64	1.166483	1.531395	2.091971
1.9	2.45	2.06	1.179864	1.973794	2.422168

16.46063	18.84472	15.16177	7.622069	14.4337	18.78968
20.91132	24.08375	19.43355	9.924581	18.49635	24.04522
16.02201	18.36819	14.7777	7.386308	14.07696	18.28246
10.05137	11.57518	9.327719	4.683798	8.888656	11.51414
15.85299	18.28119	14.76566	7.59949	14.04828	18.27532
20.56783	23.60403	19.00353	9.550073	18.09826	23.51283



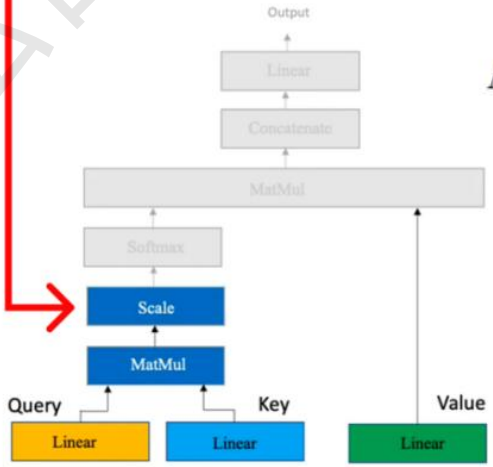
$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

16.46063	18.84472	15.16177	7.622069	14.4337	18.78968
20.91132	24.08375	19.43355	9.924581	18.49635	24.04522
16.02201	18.36819	14.7777	7.386308	14.07696	18.28246
10.05137	11.57518	9.327719	4.683798	8.888656	11.51414
15.85299	18.28119	14.76566	7.59949	14.04828	18.27532
20.56783	23.60403	19.00353	9.550073	18.09826	23.51283

$\sqrt{d_k}$ where d is 5 (dimension of embedding vector)

=

3.3	3.77	3.03	1.52	2.89	3.76
4.2	4.82	3.89	1.98	3.7	4.81
3.2	3.67	2.96	1.48	2.82	3.66
2	2.32	1.87	0.94	1.78	2.3
3.2	3.66	2.95	1.52	2.81	3.66
4.1	4.72	3.8	1.91	3.62	4.7



$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

请勿私自上传本课件

Step 1 (Performing Single Head Attention)

Step 2 (Performing Multi-Head Attention)

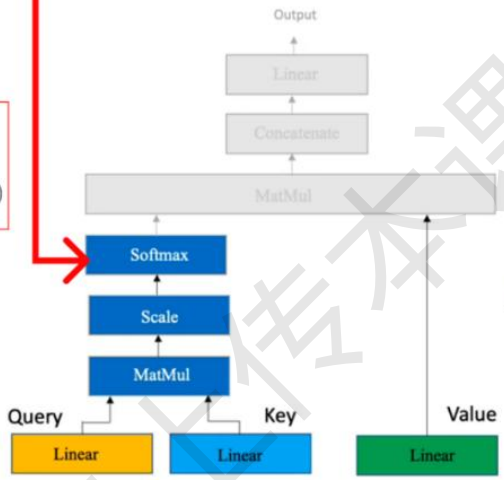
3.3	3.77	3.03	1.52	2.89	3.76
4.2	4.82	3.89	1.98	3.7	4.81
3.2	3.67	2.96	1.48	2.82	3.66
2	2.32	1.87	0.94	1.78	2.3
3.2	3.66	2.95	1.52	2.81	3.66
4.1	4.72	3.8	1.91	3.62	4.7

$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_i)}$$

$$\frac{\exp(3.3)}{\exp(3.3) + \exp(3.77) + \exp(3.03) + \exp(1.52) + \exp(2.89) + \exp(3.76)}$$

=

0.17	0.33	0.235	0.07	0.27	0.88
0.16	0.36	0.22	0.04	0.24	0.95
0.17	0.33	0.237	0.07	0.28	0.87
0.18	0.28	0.238	0.12	0.29	0.63
0.17	0.33	0.236	0.07	0.28	0.87
0.16	0.36	0.222	0.04	0.24	0.95



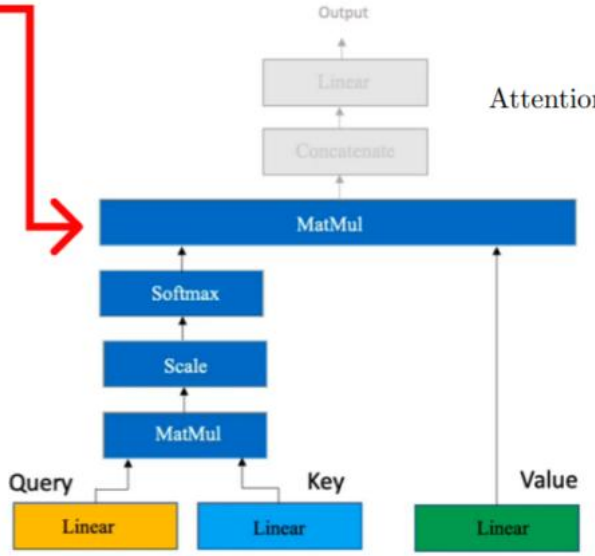
$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

$\text{softmax}\left(\frac{Q.K^T}{\sqrt{d_k}}\right)$ 将结果矩阵与值矩阵相乘

resultant matrix							Value		
0.17	0.33	0.23	0.07	0.27	0.88		3	3.4	2.06
0.16	0.36	0.22	0.04	0.24	0.95		3.97	3.7	2.84
0.17	0.33	0.24	0.07	0.28	0.87	X	2.64	2.6	2.24
0.18	0.28	0.24	0.12	0.29	0.63		2.46	1.3	1.49
0.17	0.33	0.24	0.07	0.28	0.87		3.02	2.7	2.19
0.16	0.36	0.22	0.04	0.24	0.95		3.93	3.6	2.78

=

6.89	6.4	4.96
7.07	6.57	5.08
6.86	6.37	4.94
5.89	5.43	4.24
6.84	6.34	4.92
7.08	6.59	5.09

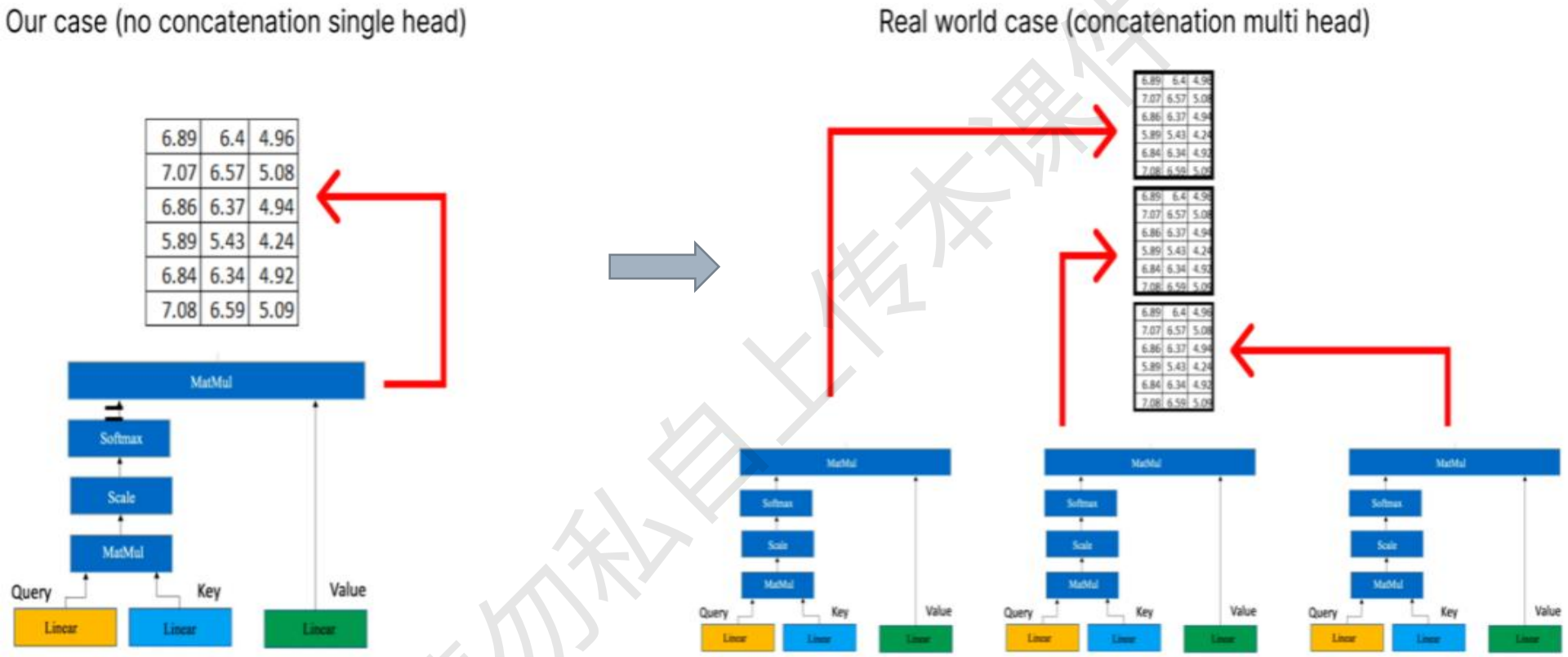


$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

清华大学出版社 课件

Step 1 (Performing Single Head Attention)

Step 2 (Performing Multi-Head Attention)



多头注意力 级联拼接

请勿私自上传本课件

生成式计算与大模型应用

1.1 ChatGPT与大模型简介

1.2 Transformer与大模型计算原理

1.3 Langchain与大模型开发框架*

请勿私自上传本课件

1.3 Langchain与大模型开发框架*

Langchain简介

简化LLM集成开发需求&提供成套工具和接口

2018

深度学习走向预训练

2018年，OpenAI和Google分别发布了GPT-1与BERT大模型，意味着预训练大模型成为自然语言处理领域的主流。

2022

大模型+大数据+大算力

2022 年底，由 OpenAI 发布的ChatGPT 引发了社会的广泛关注。ChatGPT能够通过自然语言交互完成多种任务，具备了多场景、多用途、跨学科的多任务处理能力。

至今

以GPT为代表的大模型竞赛

以 ChatGPT 为代表的大模型技术可以在经济、法律、社会等众多领域发挥重要作用，引发了大模型的发展热潮。

LangChain是一个专为大语言模型（LLM）设计的应用程序开发框架。不仅支持开发者快速上手，还提供了从开发到部署的全流程支持，使得构建基于LLM的应用程序变得更加高效快捷。

LangChain

应用场景

新闻撰写

langchain+传媒可以实现智能新闻写作，降低新闻的生产成本

脚本创作

langchian+影视可以拓宽创作素材，开拓创作思路，激发创作灵感，提升作品质量

客户服务

langchain+营销可以打造虚拟客服，助力产品营销

教育辅导

langchain+教育可以赋予教育教材新活力，让教育方式更个性化、更智能

金融决策

langchain+金融可以帮助金融机构降本增效，让金融服务更有温度

医疗咨询

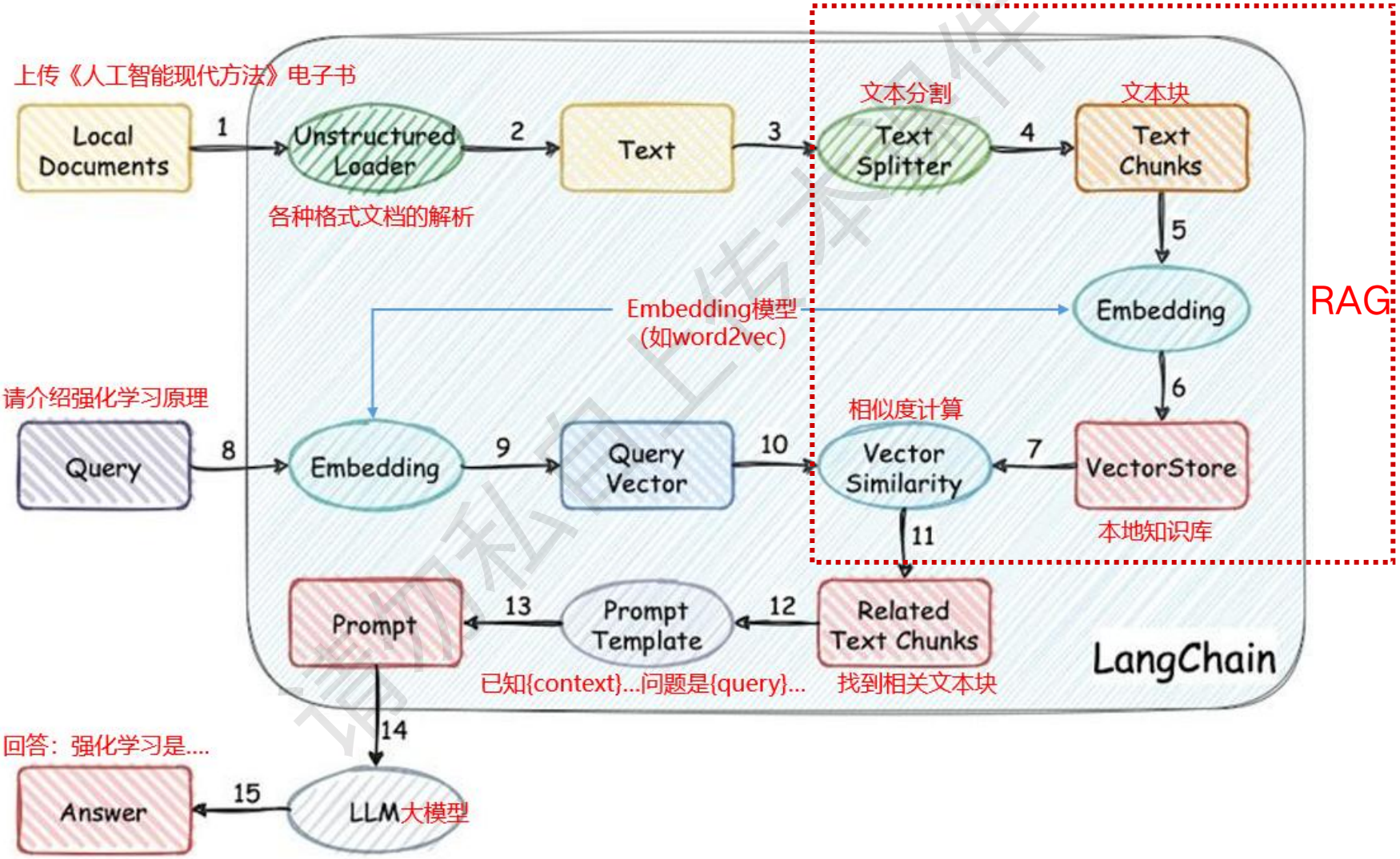
langchain+医疗可以赋能医疗机构诊疗全过程

请勿私自上传本课件

1.3 Langchain与大模型开发框架*

Langchain技术架构

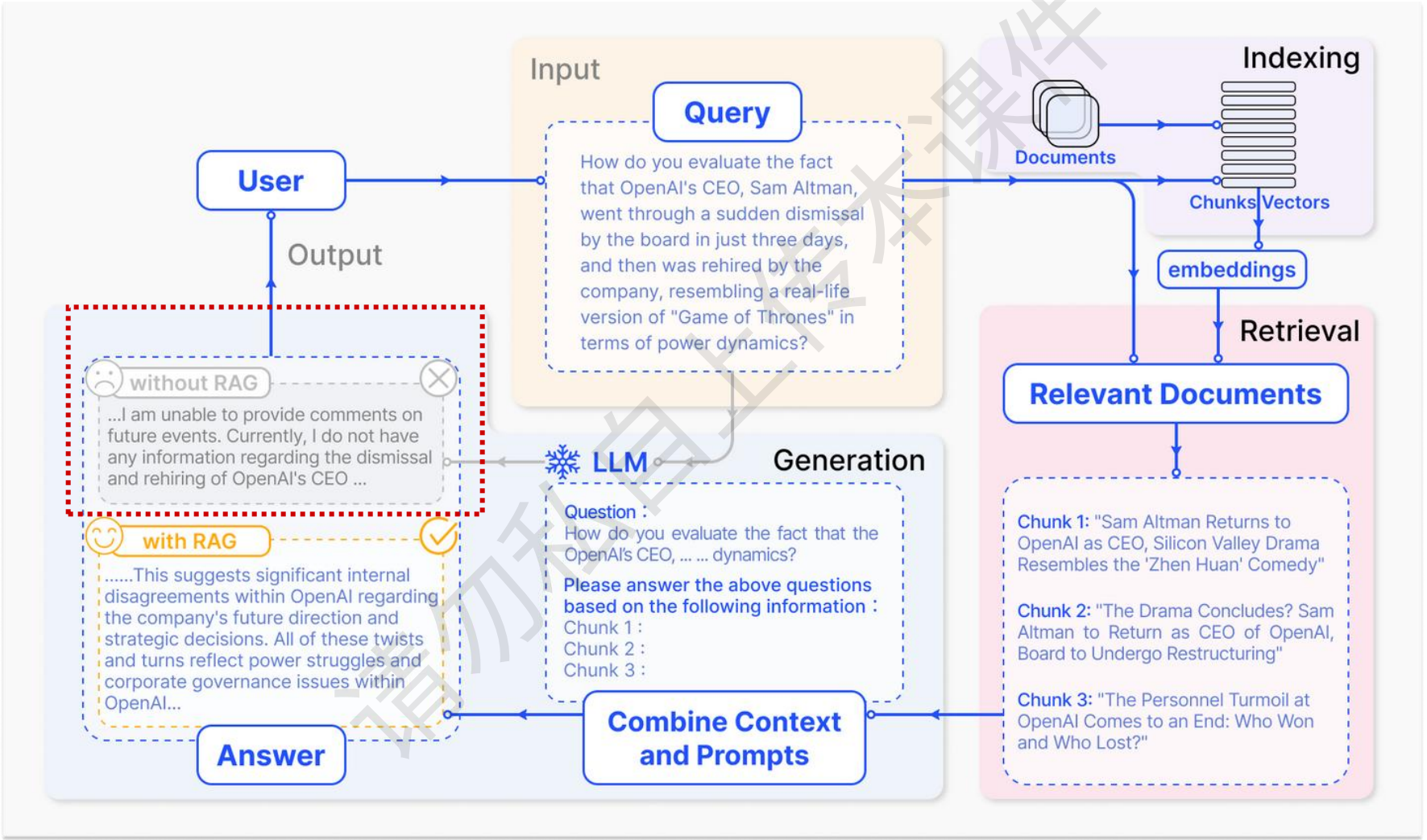
Langchain 核心技术流程》：



1.3 Langchain与大模型开发框架*

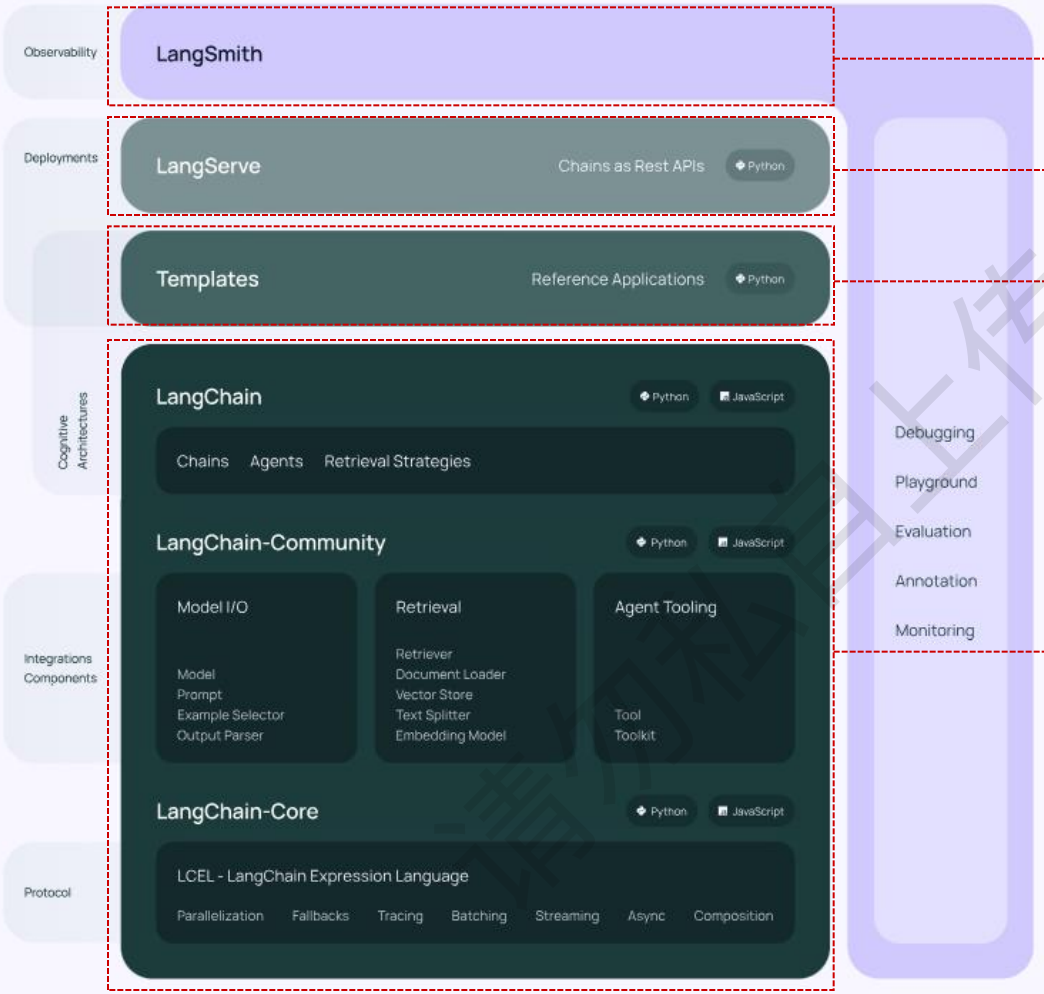
Langchain技术架构

为什么需要RAG？



1.3 Langchain与大模型开发框架*

Langchain技术架构 组件工具



LangSmith 是开发平台，方便开发人员进行调试、测试、评估、监控等。

LangServe提供了用来将 LangChain 部署为 REST API 服务的工具，以供应用的其他模块调用。

LangChain Templates提供了一组用于简化部署的参考架构，方便构建基于 LLM 的各种应用任务。

LangChain Libraries是 LangChain 中最核心的部分被分为 3 个核心的部分：

- **LangChain**包括组成应用认知架构的各个部分，如 Chains、Agents 以及 检索策略（Retrieval Strategies）等。
- **LangChain-Community**所有用来与第三方集成的功能，都会放到 LangChain-Community 中，方便构建 LLM 应用并提升集成能力。
- **LangChain-Core**包含 LangChain 的基本抽象和 LCEL（LangChain Expression Language）。

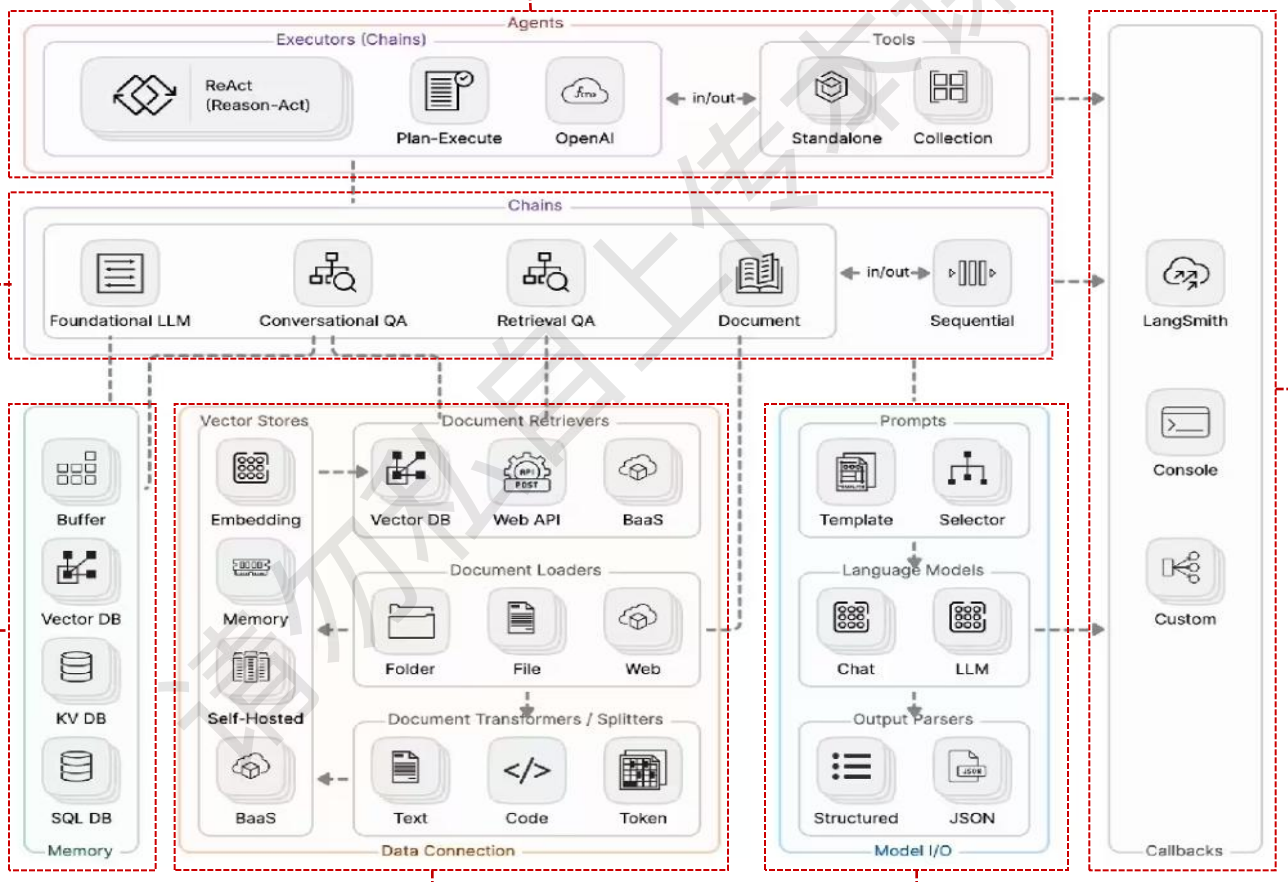
提供构建基于 LLM 的 AI 应用所需的各种组件与工具

1.3 Langchain与大模型开发框架*

Langchain核心模块

LangChain可以方便地与外部其他组件进行交互以满足面向特定领域 LLM 应用的需求，主要分为6个模块

Agents根据高级指令选择使用哪些工具（比如调用小模型）



Chains构建调用序列

Memory在链的运行之间持久化应用程序状态

Callbacks回调系统，用来支持 LLM 应用的日志记录、监控、Streaming 处理，等等

RAG本地知识融合与增强检索

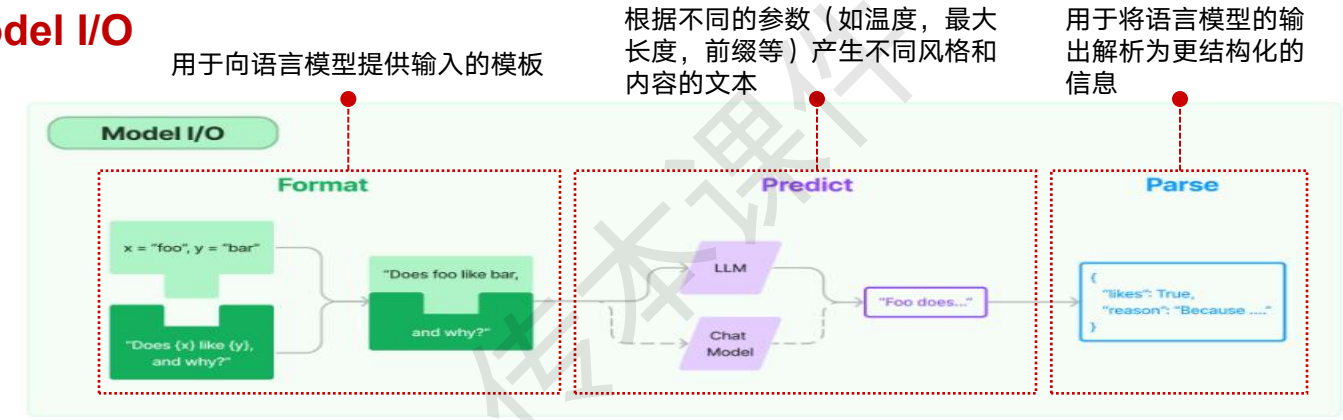
Model/IO用于与 LLM 语言模型交互

1.3 Langchain与大模型开发框架*

Langchain核心模块

LangChain 模块：Model I/O

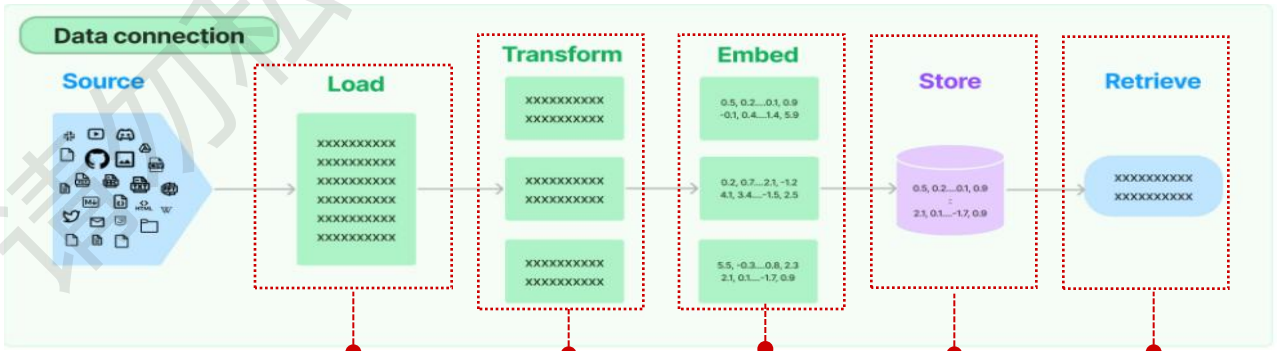
Model I/O的核心思想是**利用大模型强大的自然语言理解和生成能力**，将任何形式的输入转换为**自然语言**，然后将自然语言输入到模型中，再将生成结果**转换为任何形式的输出**。



LangChain 模块：Retrieval

LangChain 通过 Retrieval 模块来**处理有关数据连接的功能**，比如常用的 **RAG**（Retrieval Augmented Generation）架构能够增强 LLM 的表达。

LangChain 基于 RAG 实现的 Retrieval 模块的基本流程，如右图所示：



请勿私自上传本课件

1.3 Langchain与大模型开发框架*

Langchain核心模块

LangChain 模块：Chains

Chain可以让程序按照**一定的顺序和逻辑**来执行不同的任务

LLMChain

用于在语言模型周围添加一些功能的简单Chain

RouterChain

用于创建一个动态选择下一个要使用的Chain的Chain的范式

SequentialChain

用于将多个Chain连接起来，形成一个执行一些特定场景的管道的Chain

TransformChain

用于创建一个通用的转换Chain，它可以对输入进行一些预处理或后处理

LangChain 模块：Memory

Memory 模块主要用来**支持读和写**两个基本的操作。在与 LLM 交互过程中需要能够即时地**获取、保存对话信息**

Conversation Buffer

用于存储对话中的所有消息和元数据的内存

Buffer Window Entity

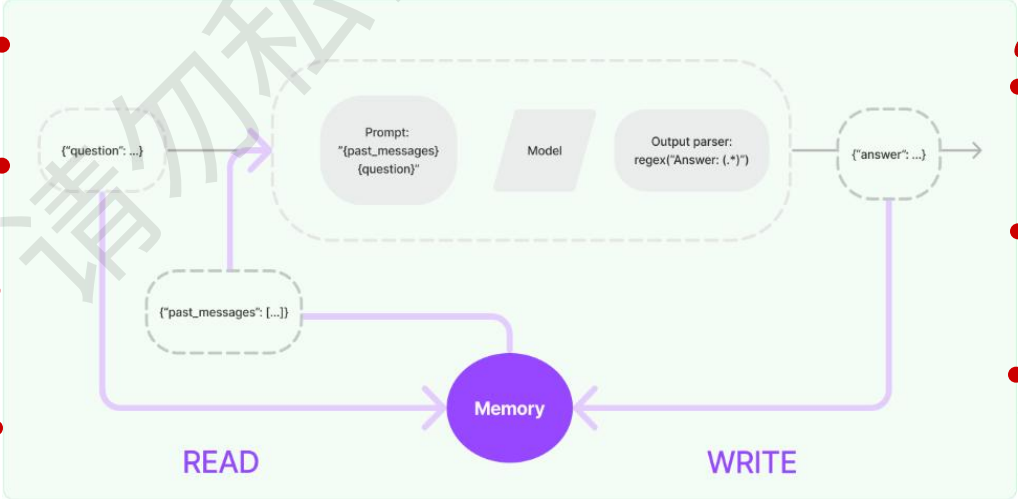
用于从对话缓冲区中提取特定窗口大小内的实体的内存

Conversation Knowledge Graph

用于构建和更新对话中涉及的实体和关系的知识图谱的内存

Conversation Summary

用于生成对话的摘要的内存



Conversation Summary Buffer

用于从对话缓冲区中提取特定窗口大小内的实体的内存

Conversation Token Buffer

用于存储对话中的所有单词和标点符号的内存

Vector Store

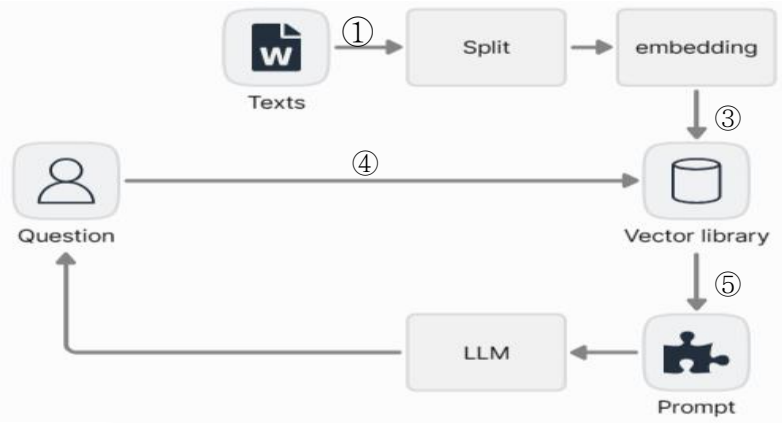
用于将对话中的文本或实体转换为向量表示，并进行相似度计算或聚类分析的内存

请勿私自上传本课件

1.3 Langchain与大模型开发框架

Langchain开发实例

langchain与 RAG 的问答　索引&检索生成



索引

- 加载：首先加载需要的数据。
- 拆分：文本分离器将大块分成更小的块。
- 存储：存储和索引数据的向量化后的结果，以便以后可以搜索向量。

检索和生成

- 检索：给定用户输入，对问题进行拆分，并从向量库中进行相似度搜索
- 拼接：利用prompt模板对问题和检索结果进行拼接
- 生成： 包含问题和检索数据的提示，送入ChatModel / LLM。

```python
import getpass
import os

os.environ["OPENAI_API_KEY"] = getpass.getpass()
```
#链接API

```python
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-3.5-turbo-0125")
```
#接入模型

```python
# Load, chunk and index the contents of the blog.
loader = WebBaseLoader(
    web_paths=("https://lilianweng.github.io/posts/2023-06-23-agent/",),
    bs_kwargs=dict(
        parse_only=bs4.SoupStrainer(
            class_=("post-content", "post-title", "post-header")
        )
    ),
)
docs = loader.load()
```
#加载需要的数据：txt、pdf、docx等

```python
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
splits = text_splitter.split_documents(docs)
vectorstore = Chroma.from_documents(documents=splits, embedding=OpenAIEmbeddings())
```
#对文本进行分块　chunk_size是分割大小 chunk_overlap是字符重叠

#向量存储

#相似度检索

```python
# Retrieve and generate using the relevant snippets of the blog.
retriever = vectorstore.as_retriever()
prompt = hub.pull("rlm/rag-prompt")
```
#prompt拼接

```python
def format_docs(docs):
    return "\n\n".join(doc.page_content for doc in docs)


rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)
```
#定义链，从而允许各组件通过管道组合在一起

```python
rag_chain.invoke("What is Task Decomposition?")
```

回答:'Task decomposition is a technique used to break down complex tasks into smaller and simpler steps. It can be done through prompting techniques like Chain of Thought or Tree of Thoughts, or by using task-specific instructions or human inputs. Task decomposition helps agents plan ahead and manage complicated tasks more effectively.'

请勿私自下载课件

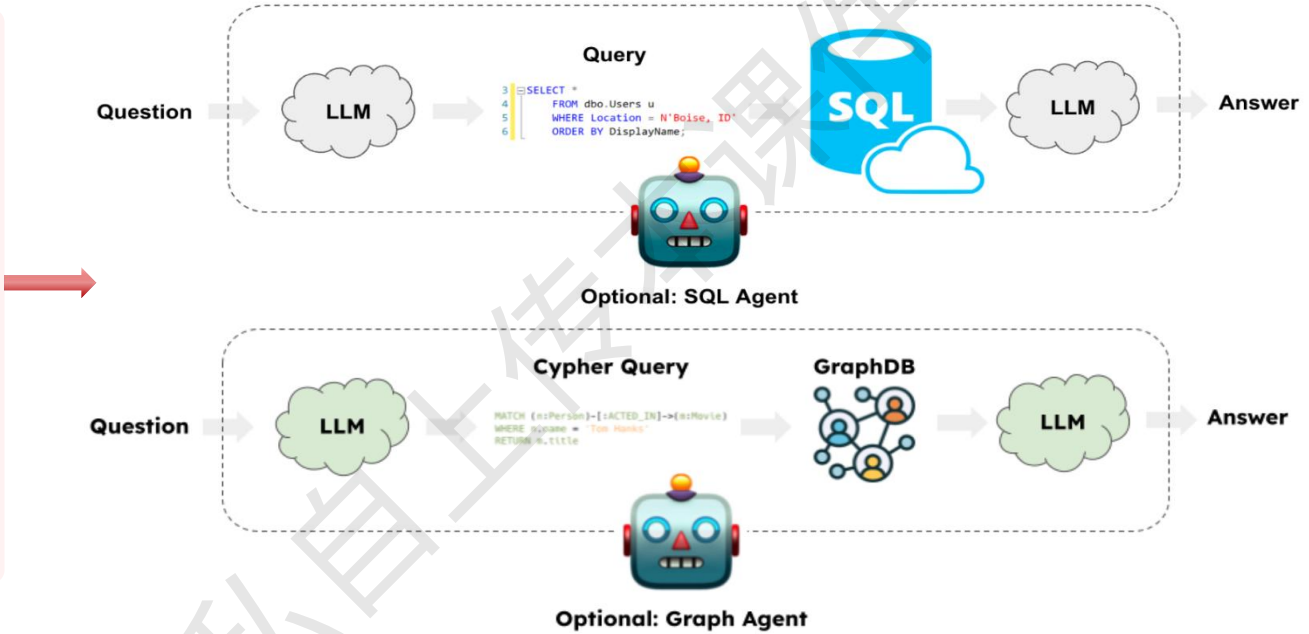
1.3 Langchain与大模型开发框架

Langchain开发实例　基于SQL与知识图谱的问答　各类数据库的代理

Agent

通过Agents，可以让大语言模型具有主动性和智能性。

Agent可以动态的帮我们选择和调用 Chain 或者已有的工具，实现面向多种应用场景的智能任务适配。



SQL 链和代理的步骤

- 将问题转换为 SQL 查询：模型将用户输入转换为 SQL 查询。
- 执行 SQL 查询：执行 SQL 查询。
- 回答问题：查询结果输入模型使用。

请勿私自上传课件

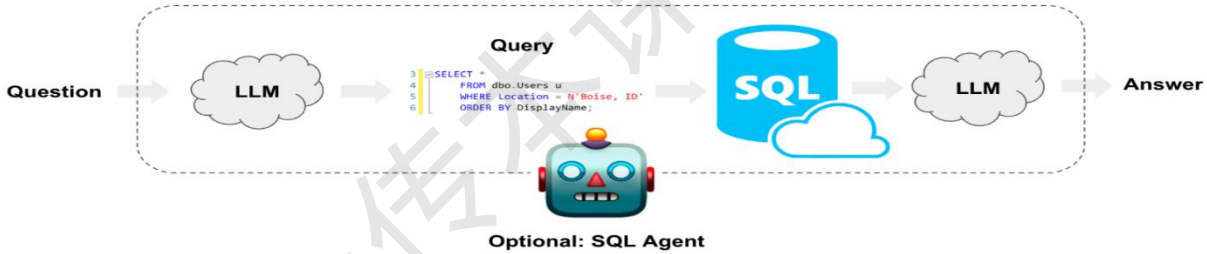
1.3 Langchain与大模型开发框架

Langchain开发实例　基于SQL与知识图谱的问答　各类数据库的代理

基于SQL与知识图谱的问答允许我们提出关于 SQL 数据库中数据的问题 输出自然语言答案。

SQL 链和代理的步骤

- 将问题转换为 SQL 查询：模型将用户输入转换为 SQL 查询。
- 执行 SQL 查询：执行 SQL 查询。
- 回答问题：查询结果输入模型使用。



#SQL 链或代理的第一步是获取用户输入并将其转换为 SQL 查询。

```
from langchain.chains import create_sql_query_chain
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
chain = create_sql_query_chain(llm, db)
response = chain.invoke({"question": "How many employees are there"})
response
```

回答：

```
'SELECT COUNT(*) FROM Employee'
```

#执行 SQL 查询

```
from langchain_community.tools.sql_database.tool import QuerySQLDataBaseTool

execute_query = QuerySQLDataBaseTool(db=db)
write_query = create_sql_query_chain(llm, db)
chain = write_query | execute_query
chain.invoke({"question": "How many employees are there"})
```

回答：

```
'[(8,)]'
```

```
from operator import itemgetter

from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import PromptTemplate
from langchain_core.runnables import RunnablePassthrough

answer_prompt = PromptTemplate.from_template(
    """Given the following user question, corresponding SQL query, and SQL result, answer the user question.

Question: {question}
SQL Query: {query}
SQL Result: {result}
Answer: """
)

answer = answer_prompt | llm | StrOutputParser()
chain = (
    RunnablePassthrough.assign(query=write_query).assign(
        result=itemgetter("query") | execute_query
    )
    | answer
)

chain.invoke({"question": "How many employees are there"})
```

#回答问题

#prompt模板

#现在我们已经有了自动生成和执行查询的方法，我们只需要将原始问题和 SQL 查询结果结合起来即可生成最终答案。

回答：

```
'There are 8 employees.'
```

请勿私自上传课件

期末考试安排

任课教师	考试时间	考试地点
马冲	2026-01-09 星期五 第3场	X2216 犀浦
马冲	2026-01-09 星期五 第3场	X2217 犀浦

考试时间：

- 第一场：9:00-11:00
- 第二场：14:30-16:30
- **第三场：19:00-21:00**

考试时间：2026年1月9号 19:00-21:00

考试地点：X2216 和 X2217

请勿私自上传本课件

期末考试试卷题型

一、填空题（10-15分，每空1分）

基本概念

二、名词解释（15-20分，3-4个名词）

算法、模型的概念解释

三、论述题（20分，2道题）

算法解释、算法步骤分析

四、分析题（25分，2-3道题）

算法的具体计算

五、综合题（25分，2-3道题）

模型的设计、前沿技术的论述

清华出自主讲大课件